SEEC: Memory Safety Meets Efficiency in Secure Two-Party Computation

Full Version*

Henri Dohmen
henri.dohmen@stud.tu-darmstadt.de
Technical University of Darmstadt
Darmstadt, Germany

Robin Hundt
robin.hundt@encrypto.cs.tu-darmstadt.de
Technical University of Darmstadt
Darmstadt, Germany

Nora Khayata
khayata@encrypto.cs.tu-darmstadt.de
Technical University of Darmstadt
Darmstadt, Germany

Thomas Schneider
schneider@encrypto.cs.tu-darmstadt.de
Technical University of Darmstadt
Darmstadt, Germany

ABSTRACT

Secure Multi-Party Computation (MPC) allows multiple parties to perform privacy-preserving computation on their secret data. MPC protocols based on secret sharing have high throughput which makes them well-suited for batch processing, where multiple instances are evaluated in parallel. So far, practical implementations of secret sharing-based MPC protocols mainly focus on runtime and communication efficiency, so the memory overhead of protocol implementations is often overlooked. Established techniques to reduce the memory overhead for constant-round garbled circuit protocols cannot be directly applied to secret sharing-based protocols because they would increase the round complexity. Additionally, state-of-the-art implementations of secret sharing-based MPC protocols are implemented in C/C++ and may exhibit memory unsafety and memory leaks, which could lead to undefined behavior.

In this paper, we present SEEC: SEEC Executes Enormous Circuits, a framework for secret sharing-based MPC with a novel approach to address memory efficiency and safety without compromising on runtime and communication efficiency. We realize SEEC in Rust, a language known for memory-safety at close-to-native speed. To reduce the memory footprint, we develop an in-memory representation for sub-circuits. Thus, we never inline sub-circuit calls during circuit evaluation, a common issue that blows up memory usage in MPC implementations. We compare SEEC with the state-of-the-art secret sharing-based MPC frameworks ABY (NDSS'15), MP-SPDZ (CCS'20), and MOTION (TOPS'22) w.r.t. runtime, memory, and communication efficiency. Our results show that our reliable and memory-safe implementation has competitive or even better performance.

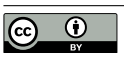
1 INTRODUCTION

Privacy-enhancing technologies (PETs) are essential to secure the fundamental right of privacy in an increasingly digital world. An important PET building block is secure multi-party computation (MPC). It allows two or more distrusting parties to jointly compute a public function on their private inputs. MPC ensures that each party learns only the output of the functionality and nothing more [75]. MPC has several applications such as privacy-preserving machine learning [61], and secure financial data analysis [14].

The seminal works on MPC in the late 1980s [8, 39, 97, 98] have been mostly of theoretical interest. However, with increasing computational and network resources together with protocol improvements, real-world deployment of MPC is becoming more and more realistic [1, 4, 33, 53]. There are two main classes of MPC protocols:
(1) Constant round protocols like Yao's garbled circuits (GCs) [98] evaluate costly symmetric cryptographic operations in the online phase. With state-of-the-art protocol optimizations [10, 62, 90], GCs provide free XORs and one AND gate has communication 1.5κ bits, κ is the symmetric security parameter. In summary, these protocols have constant rounds and low latency, but low throughput.
(2) Secret sharing-based MPC protocols like GMW [39], SPDZ [29], or ABY2.0 [81] allow to precompute as many cryptographic operations as possible in an input-independent setup phase using Oblivious Transfer (OT)-extension [5, 46], SilentOT [15], or Homomorphic Encryption [29]. This yields a very efficient online phase without any expensive cryptographic operations, but needs interaction between the parties for every layer of AND gates. The total communication can be as low as 2 bits per AND gate [81]. Secret sharing-based MPC protocols have high throughput, which makes them very well-suited for batch processing [36]. Batch processing is useful in outsourcing scenarios [23] that are commonly used in privacy-preserving machine learning (PPML) [76, 89]. Outsourcing allows utilizing efficient passively (semi-honestly) secure protocols, even if the client is malicious because the MPC protocol is run only between the outsourcing servers. However, finding non-colluding parties to run the outsourcing servers in practice is challenging, so the number of parties should be kept as low as possible [33]. Hence, many practical works, e.g., in MPC-based PPML, are in the two-party setting [16, 63, 76].

State-of-the-art implementations of general-purpose MPC protocols, including secret sharing-based protocols such as ABY [31], MP-SPDZ [54], and MOTION [16, 17], are developed in C/C++ and focus on optimizing runtime and communication efficiency. Being implemented in C++ for efficiency reasons, these frameworks can be prone to memory leakage and more general memory-safety issues such as data races, buffer overflows, and use-after-free errors,

*Please cite the conference version of this work published at ASIACCS'25 [32].



which can impact the reliability of the frameworks. For example, during our experiments with ABY [31], the framework sometimes crashed due to memory-related bugs, so we had to restart evaluation runs frequently (see §5.1.3 and Table 3).

Earlier works have presented memory-efficient and memory-safe implementations for GCs in Java [42, 44, 64, 73, 78] or Python [41]. However, Java and Python use garbage collection to free unused memory which takes a lot of runtime. Especially in secret sharing-based MPC protocols, where the operations are very lightweight in the critical online phase (mainly additions and multiplications), garbage collection significantly impacts runtime performance. All previous works on memory-efficient MPC were restricted to constant-round GC protocols, where they introduced optimizations like pipelining [6, 42, 48, 73] and compact sequential circuit representations [45, 93] (details in §6.1). However, these optimizations are not directly translatable to multi-round secret sharing-based MPC protocols, because they would increase round complexity and need careful scheduling of the online phase.

Due to the aforementioned challenges with C/C++ and languages using garbage collection, many new cryptography projects moved to developing their frameworks in the programming language Rust. NIST promotes the use of Rust as a safe language[1], Google joined the Rust foundation and moved to development with Rust[2], and first implementations of MPC protocols are realized in Rust [19, 37, 103].

Our Contributions. We present SEEC, the first memory-efficient and memory-safe framework for secret sharing-based MPC protocols implemented in Rust. Our contributions are:
(1) *Optimized memory consumption via our compact in-memory representation (§4):* We optimize memory consumption for secret sharing-based protocols through an in-memory representation that efficiently realizes sub-circuits together with Single Instruction Multiple Data (SIMD). Our sub-circuit iteration algorithms (§4.1 and §4.2.1) allow reducing the memory footprint while maintaining low round complexity during evaluation. This notion works together with SIMD, a runtime optimization known from state-of-the-art MPC frameworks [17, 31, 54]. Although SIMD itself already reduces the memory footprint, it requires that there are no data-flow dependencies between the instances of a sub-circuit. Our representation generalizes this and maintains a memory-compact representation even when sub-circuit calls have data-flow dependencies.
(2) *State-of-the-art protocols and high-level abstraction:* We implement the Boolean, Arithmetic, and mixed-mode ABY2.0 [81] protocols which use function-dependent preprocessing and offer highly efficient MPC protocols on vectors and matrices. With this, we fully include the secret sharing-based protocols of the ABY2.0 protocol family in our general-purpose framework. In addition, we implement both Boolean and Arithmetic GMW as well as conversions for mixed-mode computation. For the setup phase, SEEC supports OT-based multiplication triple generation using OT extension [5] and Silent OT [15]. Besides providing Rust implementations for OT protocols, SEEC also integrates libOTe, a highly efficient C++ library of various OT protocols. To run pre-compiled circuits, SEEC supports Boolean circuits in Bristol Fashion format [2] as well as mixed circuits in the FUSE intermediate representation (IR) [18]. FUSE IR includes a representation for sub-circuits with calls that are preserved when executing them in SEEC.
(3) *Modular MPC phases and communication abstractions:* To be both reliable and efficient, SEEC is implemented in Rust. The design abstracts the different phases of MPC via Rust's powerful type systems which combines functional programming paradigms with systems programming. This design allows us to support both function-independent and function-dependent preprocessing phases that can both be used. Additionally, it allows the addition of new phases, such as those necessary for maliciously secure protocols and general protocol advancements in the future. Using Rust's abstraction for communication channels, SEEC offers various implementations for communication between the parties with minimal communication overhead. These include in-memory communication, as well as TCP and TLS1.3 communication. This improves over the state-of-the-art design of MOTION [17], which provides generalized communication channels in C++ but with a lot of communication overhead due to the underlying serialization library that is used to realize this.
(4) *Memory safety without runtime overhead and developer-friendly programming framework:* Being implemented in Rust, SEEC provides a memory-safe and reliable framework with competitive performance. Rust's packet manager, cargo, allows researchers to develop new protocols without distractions by project management challenges that are common in C/C++ with CMake. Instead, it is possible to use SEEC as a library without fine-tuning build systems and achieve reliable and good performance.
(5) *Reproducible benchmarks of memory and runtime efficiency:* We provide a comprehensive evaluation of memory and runtime efficiency by comparing SEEC with the state-of-the-art secret sharing-based MPC frameworks ABY [31], MP-SPDZ [54], and MOTION [16, 17]. We have developed a benchmarking tool that enables easily reproducible results of all four frameworks which is of independent interest. Our evaluations show that SEEC provides memory-safety and -efficiency without sacrificing runtime efficiency. Our notion of sub-circuits is able to reduce the memory consumption of securely evaluating the MNIST neural network by factor 2×. Our framework is available open-source[3].

2 RELATED WORK

We compare SEEC with three modern implementations of secret sharing-based MPC protocols: ABY [31], MP-SPDZ [54], and MOTION [16, 17]. We provide further related work on memory-efficient garbled circuits, other MPC frameworks, and compilers in §6.

2.1 ABY

ABY [31] is a secure two-party computation framework offering mixed-protocol evaluation with passive security. It implements GCs, Boolean, and Arithmetic GMW [39] using multiplication triples [7] with efficient conversions between shares of different types. Users can manually choose different protocols for sub-functionalities.

The implementation allocates gates within contiguous memory and compactly stores the output values for the gates internally. ABY provides rudimentary support for storing and loading multiplication triples which are used in the online phase. Single Instruction Multiple Data (SIMD) operations are supported, but the framework

[1] https://www.nist.gov/itl/ssd/software-quality-group/safer-languages
[2] https://opensource.googleblog.com/2021/02/google-joins-rust-foundation.html
[3] https://encrypto.de/code/SEEC

has no functionality for sub-circuits. Loops are not supported, because ABY's internal representation is a combinational circuit. ABY provides an implementation of GMW and preprocessing with multiplication triples (MTs) but no function-dependent preprocessing.

ABY is closely related to SEEC, as its design is in many aspects similar to ABY. SEEC can be seen as a modern descendant of ABY with the capability for compact and efficient sub-circuits.

2.2 MOTION

MOTION [17] is an MPC framework for passively secure mixed-protocol evaluation among $N \geq 2$ parties of which all but one can be corrupted. It implements the multi-party BMR protocol [8] and the Boolean and Arithmetic variants of GMW using MTs [7]. Additionally, MOTION implements conversions between these protocols. The fork MOTION2NX [16] implements the ABY2.0 protocols [81] and is geared towards private inference of neural networks.

Instead of allocating gates within contiguous memory, MOTION individually allocates each gate on the heap and stores the output of gates internally within their representation. MOTION interleaves the preprocessing phase with the online phase but does not allow the storage of the preprocessing material. Similar to ABY, MOTION supports SIMD operations and furthermore supports function-dependent preprocessing. However, performing preprocessing independently of any function, i.e., precomputing multiplication triples without knowing the circuit in advance, is not straightforwardly possible.

MOTION's design differs from ABY and SEEC in several important aspects. Of particular interest is its asynchronous evaluation mode and in-memory representation of circuits. To provide sub-circuits, the representation of a circuit must be decoupled from the data that is being processed. As MOTION directly stores the values of each gate within the gate, it would require significant changes in the architecture to support sub-circuits.

2.3 MP-SPDZ

MP-SPDZ [54, 55] offers protocol implementations based on binary and arithmetic secret sharing in various security models. Contrary to ABY, MOTION, and SEEC, MP-SPDZ does not use a circuit representation for the functionality, but a high-level domain-specific language that is compiled into bytecode for protocol-specific virtual machines (VM). This bytecode representation plays a significant role in MP-SPDZ's low memory consumption for functionalities that use sequential function calls (cf. §5.1, specifically Fig. 2) since the bytecode allows for expressing functions via jump instructions, very similar to assembly. For ABY and MOTION, every function call is inlined into one big combinational circuit, which results in a very significant memory overhead.

On the other hand, one challenge with using VMs as interpreters for bytecode is more complicated parallelization, since computing the evaluation layers heavily depends on the function call sites during the execution. The biggest implication is that the VM evaluation potentially adds additional communication rounds due to the function calls. SEEC's layer iteration algorithms for sub-circuits (cf. §4.1.1) make sure that function calls do not incur additional rounds in the circuit evaluation.

Keller [55] explores loop unrolling with budgets in MP-SPDZ and the trade-offs between different unrolling budgets, the resulting bytecode size, and runtime. Notably, while no unrolling at all results in the most compact bytecode representation, choosing a good budget achieves better runtime at the expense of bytecode size. SEEC explores a similar trade-off, but in the more intricate version of keeping sub-circuits, while making sure to evaluate "as if inlined".

In a sense, SEEC combines the runtime benefits of combinational circuit-based frameworks like ABY and MOTION with a very compact representation that enjoys compact memory overhead very close to MP-SPDZ bytecode.

3 SEEC ARCHITECTURE

At a very high level, secret sharing-based MPC protocol implementations work as follows: For a public functionality represented by a circuit, secret-share the private inputs, (non-)interactively evaluate the gates in topological order on the secret shares, and finally reconstruct the outputs. Our design abstracts and modularizes the different parts of an MPC execution to achieve usable, extensible, and composable zero-overhead abstractions (§3.1). Optional functionality like single instruction, multiple data (SIMD) sub-circuits (§4.2.3) or mixed-protocol support have no cost if not used. We present an overview of SEEC's architecture in Fig. 1 and introduce its components next.

3.1 Zero-Overhead Abstractions

The zero-overhead principle stems originally from the design of C++. In its original wording it states: "What you don't use, you don't pay for" [94]. Additionally, "What you do use is just as efficient as what you could reasonably write by hand" [88]. This principle is realized in the Rust programming language through an abstraction called *traits*. Traits are the only means to realize interfaces in Rust code and can be statically as well as dynamically dispatched. Static dispatching means that the compiler generates a dedicated copy of every instance of the trait that is being used in the code, hence removing the abstraction in the compiled code completely. This concept is similar to templates known from C++. Dynamic dispatching means that whenever runtime indirection is necessary, this can still be done using the same trait. An example of this is when a concrete type is only known during runtime, so it is not possible for the compiler to remove abstraction completely. Resolving indirections will incur an extra cost during runtime but allow more flexibility if necessary. Conveniently, it is possible to use the same trait with static dispatching in some parts and dynamic dispatching in other parts and only incur the extra cost for the dynamically dispatched part. We refer to [95] for more information on the exact realisation and use-cases in Rust. SEEC's code architecture heavily relies on zero-overhead abstractions to provide modularity, extensibility, and efficiency at the same time.

SEEC Frontend. The functionality to evaluate securely can be imported from the Bristol Fashion format [2], as a FUSE IR circuit [18], or written in Rust using our high-level application programming interface (API) `Secret`. Regardless of the input format, SEEC translates the functionality into its internal circuit representation based

on a directed acyclic graph (DAG). It is also possible to directly implement circuits using this lower-level circuit description, although this is not as straightforward as the other options.

In-memory representation. The in-memory representation of SEEC realizes circuits, connections between circuits, and evaluation layers of a circuit. A circuit itself is a graph-based structure that is generic over the underlying plaintext type(s), gate type, and wire type. Sub-circuit calls are translated into circuit connections which map the caller gate IDs to the callee (i.e., the sub-circuit being called) gate IDs. Depending on what the gate IDs exactly are, we either maintain two lists of all the caller and callee gate IDs or compress the lists into ranges of gate IDs. For example, instead of storing a map of individual IDs such as `{1,2,3,4,5}`, we only store a mapping of ranges, i.e., `[1,5]`. The evaluation layers are computed using iterators that are generic over the underlying circuit type in a layer-by-layer fashion. We provide two different iteration strategies: A dynamic, on-the-fly generation during circuit execution (§4.1) and a static, compile-time precomputation (§4.2.1).

Protocols and Gates. We model `Protocol` and `Gate` as two different traits (cf. §3.1) such that every implementation of `Protocol` has an associated `Gate` type that also implements the `Gate` trait. Associated types are a feature in Rust[4], which basically encapsulate the associated type, here the type `Gate`, inside a trait, here the `Protocol` trait. Taking Boolean GMW as an example, the `BooleanGMW` implementation is connected to the `BooleanGate` concrete type. Methods that are implemented inside `Protocol` operate over the `Protocol::Gate` type that is inside the `Protocol` trait.

Communication Channels. We provide a wrapper library on top of the communication library remoc[5]. Remoc creates multiplexed channels over any ordered and reliable transport. We extend it with APIs to create TCP or in-memory channels. Integrating other transport layer protocols, e.g., QUIC [68], is feasible with some additional glue code, requiring no modifications to SEEC.

Preprocessing and Oblivious Transfer (OT). SEEC's preprocessing implementation varies depending on the selected MPC protocol. It offers dedicated interfaces for function-independent preprocessing, e.g., the generation of multiplication triples (see Appendix A.3.1), or function-dependent preprocessing, as needed for ABY2.0 [81] and FLUTE [19]. For OT, we use the Rust library ZappOT from [19].

Circuit Executor. The `Executor` component coordinates the execution of a circuit using a specific protocol implementation. As this component only depends on the generic `Protocol` interface, it can be used with any of the protocols provided by SEEC. Hence, our framework can be extended with new protocols by adding it as a dependency to a project with a single command, requiring no changes to SEEC itself. When implementing a new protocol, it is easy to build upon the existing protocols and functionality in SEEC. For example, our implementation of mixed arithmetic and Boolean GMW uses the individual protocol implementations for single-protocol gates and only adds the necessary conversion gates.

[4]https://doc.rust-lang.org/rust-by-example/generics/assoc_items/types.html
[5]https://crates.io/crates/remoc

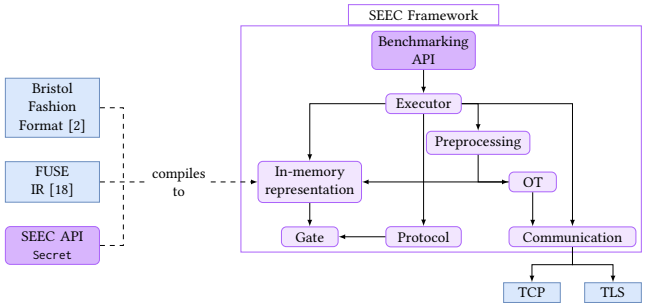


Figure 1: Architecture of the SEEC Framework

High-Level API `Secret`. To specify and use sub-circuits in a high-level fashion, one can use our API `Secret`. It provides an abstraction over the concrete sharing semantics and gates of any underlying protocol. A value of type `Secret` can be used similar to standard value types like `int` with overloaded operators such as "+" or "&". To declare a function as a sub-circuit, it suffices to add the annotation `#[sub_circuit]` above the function declaration.

Listing 1: Multi-input AND sub-circuit creation and usage using high-level API `Secret`.

```
#[sub_circuit]
fn multi_and(a: Vec<Secret>, b: Vec<Secret>)
    -> Vec<Secret> {
    a.into_iter()
        .zip(b)
        // Apply AND to each tuple of elements.
        .map(|(el_a, el_b)| {
            el_a & el_b
        }).collect()
}

// Initialize vectors a, b  and c with constants for demonstration.
let a = vec![Secret::from_const(0, false); 10];
let b = vec![Secret::from_const(0, true); 10];
let c = vec![Secret::from_const(0, true); 10];

// multi_and can be called as a simple function.
let result = multi_and(a, b);
// Calling it again will not duplicate the multi_and sub-circuit.
// Both sub-circuits are evaluated in parallel.
let second_result = multi_and(a, c);
```

We present an example using the API `Secret` in Listing 1. Here, we define a sub-circuit for multi-input AND gates called `multi_and` which we can now use with function calls. We call `multi_and` twice, once with inputs `(a,b)` and once with inputs `(a,c)`. Clearly, these two sets of inputs do not dependent on each other, so they can be evaluated in parallel with secret sharing-based MPC protocols. The iteration algorithm from §4.1 makes sure that SEEC evaluates these two sub-circuit calls in parallel.

3.2 Memory Safety

The implementation of SEEC's in-memory representation and the vast majority of the code base is written in Safe Rust[6]. Safe Rust has the following soundness property: Whatever is written in Safe

[6]https://doc.rust-lang.org/nomicon/meet-safe-and-unsafe.html

Rust cannot cause Undefined Behavior (UB). Code that could potentially lead to UB has to be marked explicitly and contained in code blocks using the `unsafe` keyword. In the context of MPC, memory bugs could lead to crashes for the honest parties even without any party aborting. This in turn could introduce security flaws and leak private information to the other party.

SEEC requires no Unsafe Rust code to operate. Uses of `unsafe` are limited to 1) the FUSE frontend's flatbuffers-based [40] deserialization of FUSE IR, and 2) optional SIMD optimizations for matrix transposition using specific hardware instructions.

FUSE Frontend. The FUSE frontend uses Google's flatbuffers [40] to deserialize FUSE IR into SEEC's in-memory representation (cf. §3.1). Flatbuffers generates code from a schema defining FUSE IR's structure. In Rust, this code generation produces `unsafe` blocks. The `unsafe` usage is confined to flatbuffers' mechanically generated code that maps FUSE IR to in-memory data structures.

Matrix Transposition. SEEC's matrix transposition algorithm includes an architecture-specific optimized variant as an alternative to the safe implementation. This version uses x86 SSE2 intrinsics for SIMD parallelization which necessitates `unsafe` blocks.

In practice, SEEC's strictly controlled `unsafe` usage leads to high operational stability. Being primarily C/C++-based, existing MPC frameworks are inherently unsafe. While other frameworks frequently crashed in our benchmarks, SEEC maintained consistent execution without memory safety violations.

4 SUB-CIRCUITS WITH SECRET SHARING-BASED MPC PROTOCOLS

To reduce the memory consumption for evaluating secret sharing-based MPC protocols, we define *sub-circuits*. This allows users of SEEC to designate parts of a circuit separately as a reusable sub-circuit. When *using* a sub-circuit, SEEC only stores the connections of gates between the main circuit and the specific sub-circuit *use*. Sub-circuit uses can be thought of as function calls in regular programming languages. However, the evaluation of a sub-circuit by a secret sharing-based MPC protocol results in an important constraint: They should not incur additional communication rounds, as the number of communication rounds can significantly impact the runtime of the online phase, especially in high-latency networks. Hence, this rules out the trivial solution — which works fine for constant-round GCs — of evaluating all sub-circuits independently of each other, as it adds rounds compared to evaluating the same circuit representation with inlined sub-circuits.

Instead, SEEC implements sub-circuits using a collection of graphs represented by efficient adjacency lists. Each unique sub-circuit is represented by one graph in this collection. Connections between individual *usages* of sub-circuits are stored externally in a data structure based on a B-Tree for fast and cache-efficient lookup [26]. To reduce the memory overhead of sub-circuit usage, we compress edges between consecutive groups of nodes into edges on ranges of nodes. During the online phase, we create an iteration object per usage of each sub-circuit. This iterator lazily computes the layers of the corresponding sub-circuit use without completely expanding it, which we refer to as dynamic layer iteration (§4.1).

Terminology. With *base-circuit* we denote a circuit object that stores its gates and wires in the form of a graph, i.e., the actual circuit definition. The *main*-circuit is the base-circuit with `ID = 0` and serves as the starting point for circuit execution. Hence, the main-circuit is *not* a sub-circuit. Any *sub*-circuit is another base-circuit with a different `ID ≠ 0`. A sub-circuit *use* is the connection between the main-circuit and a sub-circuit. We call a circuit that contains sub-circuit calls but itself is not a sub-circuit an *aggregate* circuit.

Main Idea. The main idea of our algorithms for computing the evaluation layers is to separate the representation of a sub-circuit, i.e., the nodes and edges of the graph, from the storage of the actual values during execution. This allows the evaluation of sub-circuits in parallel multiple times while having a low memory footprint and a minimal number of communication rounds, which was not possible in previous MPC frameworks. ABY [31] and MOTION [17] store the output of a gate during execution within the gate object. This is incompatible with sub-circuits, where the output of a gate is different for different call-sites, i.e., sub-circuit uses. MP-SPDZ [54] provides different possibilities of defining and introducing sub-functionalities, e.g., by defining them in bytecode, when importing Bristol Fashion circuits [2], in the high-level Python syntax, or via loop annotations. Depending on how a sub-functionality is defined and which specific optimizations are applied, it might not always happen that MP-SPDZ evaluates the overall functionality with optimal rounds. In contrast, SEEC's dynamic layer iteration (§4.1) selects the evaluation layers in a round-optimal way.

4.1 Dynamic Layer Iteration (DL)

The goal of dynamic layer iteration is to allow any sub-circuit to be used multiple times within the main-circuit without duplicating the underlying sub-circuit's base-circuit representation (inlining) which would increase memory consumption. At the same time, we aim to keep the number of communication rounds minimal. We achieve this by developing two nested iterators that compute each layer on the fly: one for computing the evaluation layers inside a sub-circuit which itself has no calls (`BaseLayerIter`, §4.1.1) and one that uses this information to compute the evaluation layers in aggregate circuits (`AggregateLayerIter`, §4.1.2). For each sub-circuit use, we create a new `BaseLayerIter` object (Listing 2) which computes the evaluation layers for the specific sub-circuit. This allows us to concurrently iterate over one base-circuit multiple times. Using the base layer iteration, the `AggregateLayerIter` object creates one global evaluation layer.

4.1.1 Base Layer Iteration. We present our iteration algorithm which lazily computes the current evaluation layer in Listing 2. The goal of the iterator is to generate the local and global evaluation layers on the fly in one iteration over the whole circuit and sub-circuits. This way, the algorithm maintains low runtime and memory overhead. The `BaseLayerIter` object contains the following data structures:

- `next_layer`: queue of gate IDs which are part of the next layer to be executed.
- `current_layer`: queue of gate IDs which contains all gates of the current evaluation layer.

- `prev_interactive`: queue of gate IDs which are all interactive gates from the previous rounds. This is used to check if their children are ready to evaluate in the current layer.
- `inputs_needed`: mapping of gate IDs to the remaining number of inputs needed to evaluate this gate. The gate ID implicitly acts as an index to access the inputs that are needed for evaluating the sub-circuit call.

The iterator returns a `Layer` object which contains all non-interactive and interactive gates of the current evaluation layer. The iteration algorithm starts with an initial set of gates, which are usually the input gates, that are added to the `next_layer`. After evaluating the gates in the current layer, we compute the next evaluation layer using the `BaseLayerIt::next` function. Although the iterator approach results in a memory cost per iterator object that is linear in the size of the underlying base-circuit, it consumes less memory than inlining the whole base-circuit for each of its uses.

`BaseLayerIter::next` *Function.* Computing the next evaluation layer during circuit execution is done via `BaseLayerIter::next`. Upon calling `next`, the queues for the next and current (empty) layers are swapped in place. After the swap, the `next_layer` queue will be empty, and `current_layer` now contains the contents of `next_layer` before the swap. If interactive gates were returned in the previous layer, i.e., they were executed in the previous layer, we check for their successor gates if all inputs are available and if so, add them into `current_layer`. We dequeue gate IDs from the front of `current_layer` until the queue is empty, where for each gate ID, we check if the corresponding gate in the base-circuit is interactive or not. When interactive gates are added to the `Layer` object to return, they are also pushed onto the `prev_interactive` queue, so that their successors are checked for the next layer. When a non-interactive gate is added to the `Layer` object, its successors are added to the current layer queue if all their inputs are available as well. A non-interactive gate may depend on previous non-interactive gates within the same layer. An interactive gate may only depend on non-interactive gates in the same layer, otherwise it would depend on other interactive gates of the same layer and hence not yet ready for evaluation. Any gate within a layer may depend on any gate of a previous layer.

Listing 2: Pseudocode for the definition and iteration of the base circuit layer iterator. This iterator lazily partitions a circuit into layers of gates in topological order.

```
class BaseLayerIter:
  graph: Graph,
  inputs_needed: [int]
  next_layer: Queue<Id>
  current_layer: Queue<Id>
  prev_interactive: Queue<Id>

  def __init__(self, i, G, I, O):
    self.graph = G
    self.inputs_needed = [len(pred(v)) for v in G.V]
    self.next_layer = Queue()
    self.current_layer = Queue()
    self.prev_interactive = Queue()

  # decrement successors' inputs_needed of v and add to queue if
  # no more inputs are needed
  def add_ready_successors(self, v, queue):
    for s in succ(v):
      self.inputs_needed[s] -= 1
      if self.inputs_needed[s] == 0:
        queue.push_back(s)

# returns next layer in topological order or None
def next(self):
  # next layer queue becomes the current layer
  swap(next_layer, current_layer)
  # current layer is empty, as we popped all elements
  # in previous iteration
  layer = Layer()

  # check previous interactive gates successors for current layer
  while v = prev_interactive.pop_front():
    self.add_ready_successors(v, inputs_needed, current_layer)

  # pop from the front of the queue until empty
  while v = current_layer.pop_front():
    if G.is_interactive(v):
      layer.push_interactive(v)
      # consider successors in **next** iteration
      self.prev_interactive.push_back(v)
    else:
      layer.push_non_interactive(v)
      # potentially add successors of non-interactive gate to
      # **current layer**
      self.add_ready_successors(v, inputs_needed, current_layer)

  if layer.is_empty():
    # we have yielded all gates and this iterator is exhausted
    return None
  else:
    # this layer can be evaluated in one round
    return layer
```

4.1.2 Aggregating layers from different calls. With `BaseLayerIter` we can so far compute the next evaluation layer for every sub-circuit *use* in a layer-by-layer fashion. Using the evaluation layers of sub-circuit uses, `AggregateLayerIter` now, in a similar fashion, computes the evaluation layers for an aggregate circuit. The idea is to aggregate all of the sub-circuit layers and evaluate them in the same round, which allows us to maintain the global minimal number of rounds. We present the algorithm of the aggregate iterator in Listing 3. `AggregateLayerIter` contains the following data structures:

- `circuits`: list of underlying `BaseCircuits` for sub-circuits and aggregate circuit.
- `call_map`: a mapping of gates for inter-circuit connections.
- `base_layer_iters`: mapping of unique `CallIds` to the underlying `BaseLayerIter` for all call. Each sub-circuit use inside the aggregate circuit has a distinct `CallId`.

Similar to `BaseLayerIter`, the `AggregateLayerIter` returns a `Layer` object which contains all non-interactive and interactive gates of the current *global* evaluation layer. First, it collects all the current layers of all base iterators which are to be evaluated now. Then it checks inside every layer if a gate is connected to a gate inside another circuit. If that is the case, the aggregate iterator resolves the inter-circuit connections using the `call_map` and schedules the connected gate for the next evaluation layer.

Listing 3: Pseudocode for the definition and iteration of the aggregate circuit layer iterator. This iterator collects all existing layers and aggregates them into one final layer for evaluation.

```
class AggregateLayerIter:
  circuits: [BaseCircuit],
  call_map: CallMap,
  base_layer_iters: HashMap<CallId, BaseLayerIter>

  def __init__(self, circuits, call_map):
    self.circuits = circuits
    self.call_map = call_map
    base_layer_iters[0] = BaseLayerIter(*circuits[0])

  def next(self):
    layers = []
    for call_id, iter in self.base_layer_iters:
      if layer := iter.next() is not None:
        layers.push((call_id, layer))
      # remove layer if exhausted
      if iter.is_exhausted():
        self.base_layer_iters.remove(iter)

    # for each gate in layers, check if connected to another circuit.
    # If yes, update BaseLayerIter state of corresponding BaseCircuit
    # call to continue with these gates in next iteration
    for from_call_id, layer in layers:
      for gate in layer:
        for to_call_id, outgoing in self.call_map.outgoing(
              from_call_id, gate):
          self.base_layer_iters[to_call_id].next_layer.push_back(
                outgoing)
```

4.2 Further Memory Optimizations

To improve memory and runtime performance, SEEC implements optimizations on the level of the circuit representation, the setup phase, and the online phase.

4.2.1 Static Layer Representation (SL). In §4.1, we described our approach for dynamically computing the layers of each sub-circuit use. Our evaluation (§5) revealed a higher than anticipated overhead of this dynamic approach. To further improve the memory consumption of sub-circuits and reduce the runtime cost resulting from the intermediate representation (IR), SEEC offers a second circuit representation. This representation is composed of static layers and computed from the dynamic representation in an offline compilation phase. We reuse the dynamic layer iteration API to precompute the layers of every sub-circuit use and store them in a deduplicated way. In this context, deduplication means that if two call sites have the same layer partitions, we only store this partitioning once and not per each call site. Only if, due to different data flows, the layer partitions end up looking different for two call sites do we create and store two different layer partitions. By precomputing the layers, we improve the runtime of the online phase at the expense of performing the precomputation. Additionally, precomputation allows us to choose a more compact circuit representation for evaluation that does not include node successor information and results in better memory efficiency. The dynamic layer representation (§4.1) would need this successor information for computing the layers ad hoc.

4.2.2 Interleaved Setup using Stored Multiplication Triples (IS). By leveraging our generic function-independent and -dependent setup interfaces, we provide an API to precompute multiplication triples (MTs) in batches, store them to a file, and use them during the online phase. This circumvents the need to keep all MTs for the entire circuit in memory. While we currently use the interleaved setup feature to only read precomputed MTs, this is easily extensible to compute the preprocessing data interleaved with the online phase.

4.2.3 Single Instruction, Multiple Data (SIMD). Building on prior work that showed the clear relevance of SIMD operations [17, 31, 74, 91], we apply this concept to our sub-circuit approach. In contrast with a scalar circuit that operates on individual shared values, SIMD operations process a vector of shares. By applying SIMD at a higher level than an individual gate or instruction, we keep the evaluation of scalar circuits memory-efficient. Concretely, we store the outputs of a scalar circuit in a flat memory region, bit-packed in the case of Boolean GMW, whereas the output of gates in a SIMD circuit are individually allocated. Supporting gate-level SIMD would increase the memory consumption of executing scalar circuits.

4.2.4 Early Freeing of SIMD Gate Outputs (FG). The output of a gate in a SIMD sub-circuit is an individually allocated vector (§4.2.3), which, in turn, means that we can also individually deallocate them. Our free gates (FG) optimization tracks the uses of gate outputs by successor gates. If the last successor of a gate is added to a layer, we mark the associated storage for deallocation. While waiting on the network in the interactive part of a layer, we free the memory of these gate outputs. The effect of this is similar to storing gate outputs in reusable registers [42, 54]. Currently, this optimization is only done when executing a SIMD sub-circuit with the dynamic layer representation.

5 EVALUATION

We performed benchmarks and comparisons of SEEC with the state-of-the-art MPC frameworks ABY [31], MOTION [17], and MP-SPDZ [54], comparing their memory consumption, runtime, and communication. We have designed and implemented SEEC with benchmarking in mind and extended the frameworks ABY [31] and MOTION [17] with insecure preprocessing implementations for a fair comparison. We implemented a dedicated distributed benchmark executor, that builds the compared frameworks, evaluates user-specified benchmarks either locally or remotely, sets up simulated networks, restricts the number of CPU cores, and parses and aggregates the benchmark results. This ensures the reproducibility and transparency of our benchmarking setup. Listing 4 shows an example configuration for the benchmark executor that specifies command line parameters for the different frameworks.

Listing 4: Example configuration for the benchmarking tool.

```
net_settings = ["RESET", "LAN", "WAN"]
repeat = 5

[[bench]]
framework = "SEEC"
target = "bristol"
tag = "seec_aes_ctr_no_setup"
compile_flags = ["../../../circuits/advanced/aes_128.bristol"]
flags = ["--insecure-setup"]
```

```
# restrict number of cores for taskset.
# 0 means no restriction.
cores = [0,1]
[bench.compile_args]
--simd = ["1", "10", "100", "1000", "10000", "100000", "1000000"]

[[bench]]
framework = "MOTION"
tag = "motion_aes_no_setup"
target = "aes128"
flags = ["--insecure-setup"]
cores = [0,1]
[bench.args]
--num-simd = ["1", "10", "100", "1000", "10000", "100000",
      "1000000"]

[[bench]]
framework = "ABY"
tag = "aby_aes_setup"
target = "bristol"
flags = ["-c", "../../bin/circ/aes_128.aby", "-d", "256"]
[bench.args]
-n = ["1", "10", "100", "1000", "10000", "100000"]

[[bench]]
framework = "MP-SPDZ"
tag = "mp_spdz_aes_no_setup"
target = "aes_circuit"
compile_flags = ["1", "10", "100", "1000", "10000", "100000"]
flags = ["-F"]
cores = [0,1]
```

Benchmarking Setup. We run our benchmarks on two servers, each with an Intel Core i9-7960X CPU with 16 physical and 32 logical cores, a clock speed of 2.8 GHz, and a 22 MB cache. The servers have 128 GB of DDR4 RAM and are connected via a 10 Gbit/s Ethernet connection. We execute the benchmarks using the following network settings:

- The **Fast-LAN** setting is the unrestricted 10 Gbit/s connection between the servers, with an average round trip time (RTT) of 0.25 ms.
- In the **LAN** setting, we use the `tc` program to restrict the bandwidth to 1 Gbit/s and add a send delay of 0.5 ms for each party (1.25 ms RTT).
- Representative of a slow intercontinental connection is the **WAN** setting, with a bandwidth of 100 Mbit/s and an added delay of 50 ms (100.25 ms RTT).

To accurately record the maximum heap usage of the different frameworks, we use the memory profiler `heaptrack`[7]. We run each benchmark configuration five times and report the mean.

Evaluated Circuits. To determine the performance characteristics of the evaluated frameworks, we benchmark a small number of circuits comprehensively with different input sizes, CPU core counts, and network settings. We construct a circuit from sequentially chained AES-128 evaluations[8] to benchmark the memory-savings enabled by sub-circuits for a sequential functionality. To evaluate SIMD sub-circuits and our optimizations (§4.2), we execute parallel AES and SHA-256 circuits. N_S denotes the number of sequential circuits, and N_P is the number of parallel circuits. We increase these exponentially to gain insight into how the different frameworks perform across a wide range of circuit sizes.

[7] https://github.com/KDE/heaptrack

[8] The AES-128 Bristol Fashion circuit we use as a basis includes the key schedule.

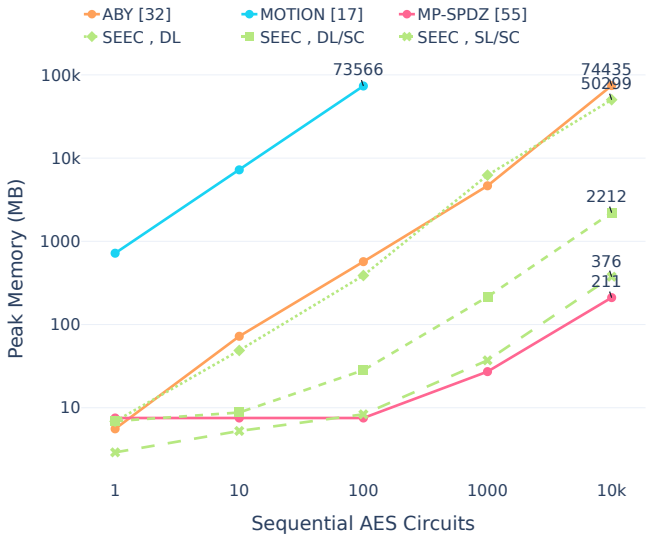


Figure 2: Peak memory for the sequential AES benchmark. Evaluated excluding setup in the LAN setting with 32 cores.

5.1 Memory Usage

The main goal of SEEC is reducing memory consumption for secret sharing-based MPC protocols at scale. Hence, in this section, we investigate the peak memory consumption of ABY [31], MOTION [17], and MP-SPDZ [54] and compare those with the memory needed by SEEC when using sub-circuits (§5.1.1 and §5.1.2) or SIMD with optimizations (§5.1.3).

5.1.1 Memory-efficiency via sub-circuits (Fig. 2). Our evaluation of the sequential AES circuit shows substantial differences in peak heap memory consumption of the different frameworks.

For SEEC, peak memory consumption reduces when using sub-circuits (DL/SC from §4.1) by factor 22× compared to a single circuit (DL from §4.1). When using the static layer representation (SL/SC from §4.2.1), this factor increases to 134× compared to the dynamic representation without sub-circuits (for N_S = 10k).

MOTION consumes the most memory by a wide margin. The high memory overhead of MOTION is due to its use of `boost::fibers`[9] in its implementation of asynchronous evaluation. For each gate, MOTION creates one `fiber` with a fixed stack size of 14KB, which is the major part of memory consumption in MOTION. For N_S = 100 sequential AES evaluations, a circuit with only 3.6 million gates, the memory consumption is over 70GB. This is almost 10,000× larger than that of the best framework MP-SPDZ (7.5MB).

For ABY, the memory consumption is very similar to SEEC without sub-circuits, which confirms that SEEC does not introduce additional baseline memory overhead compared to ABY. This is also what one would expect as some implementation parts of SEEC were inspired by ABY (§2.1). In addition, introducing sub-circuits results in an overall lower memory consumption than ABY.

[9] Fibers are userland-threads with lower overhead than native OS threads.

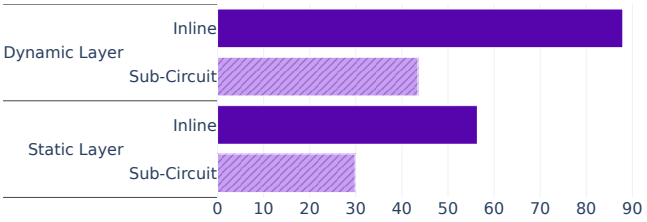


Figure 3: Peak memory consumption of SEEC of the online phase for executing the MNIST neural network of [76] with mixed-protocol (A+B) GMW with dynamic (§4.1) and static layer iteration (§4.2.1).

While SEEC needs the lowest amount of memory for small circuits ($N_S \in \{1, 10\}$ sequential circuits), MP-SPDZ is more memory-efficient for larger circuits. MP-SPDZ performs better than SEEC in this benchmark, as it supports loops, which allow a more compact representation and reuse of virtual machine (VM) registers.

5.1.2 Use-case: Neural Networks (Fig. 3). Neural networks often comprise multiple layers of matrix multiplication. For repeating matrix multiplications with identical input dimensions, such as in the MNIST neural network benchmarked in [76], we can use SEEC's sub-circuits to reduce the size of the circuit and the memory consumption of the evaluation. We benchmark our implementation of the mixed-protocol (A+B) GMW implementation with the MNIST neural network from [76] as implemented in FUSE [18]. Our evaluation in Fig. 3 shows the difference in memory consumption between using sub-circuits (§4) and inlining every occurrence of a matrix multiplication and activation function, i.e., using a single circuit and no sub-circuits. For both the dynamic layer (§4.1) and static layer representation (§4.2.1), the use of sub-circuits decreases the peak memory needed to evaluate the network by up to 2×. We anticipate larger differences for more complex neural networks.

5.1.3 Memory efficient-SIMD (Fig. 4 and Table 1). The GMW implementation of [91] emphasized the impact of SIMD gates on the memory consumption of the Boolean GMW protocol. In Fig. 4, we show the relation between the peak memory consumption and the number of parallel circuits N_P, i.e., SIMD size. Depending on the framework, the memory consumption of evaluating up to $N_P \in \{100, 1\text{k}\}$ parallel AES circuits is close to that of evaluating a single circuit. Only for massively parallel circuits, the memory consumption increases linearly with N_P for MOTION, ABY, and SEEC. MP-SPDZ shows irregular memory consumption that rapidly increases for $N_P > 1\text{k}$ parallel circuits and does not allow evaluating a SIMD size of $N_P > 10k$. Similarly, ABY could not evaluate the largest number of parallel circuits due to a segmentation fault.

Our evaluation shows that SIMD operations and sub-circuits significantly improve the memory efficiency of secret sharing-based MPC. With SEEC's notion of SIMD sub-circuits and optimizations (§4.2), we can evaluate larger circuits than ABY or MP-SPDZ and significantly improve the memory consumption compared to MOTION. For $N_P = 10^6$, SEEC consumes 15.54× less memory than MOTION. The amortized peak memory consumption is 0.2 bits per gate. Our early gate output deallocation optimization (FG in §4.2.4) improves peak memory consumption by a factor of 2.44×

Table 1: Peak memory consumption in MB of N_P parallel SHA circuit evaluation including setup. Best results are marked in bold. SEEC's IS (§4.2.2) uses precomputed MTs.

Framework	N_P Optim.	100	10,000	500,000
MOTION [17]		3,381.2	74,332.2	–
SEEC	SL	189.6	18,524.2	–
	DL/IS/FG	**27.6**	**37.5**	**361.8**

over the SEEC baseline. This improvement factor increases to 10.9× when combining the deallocation optimization with the usage of interleaved precomputed setup (IS in §4.2.2).

Our evaluation for parallel SHA-256 circuit evaluations, including setup, shows a similar picture (see Table 1), with an even higher improvement factor of up to 1,983× when comparing SEEC, with precomputed interleaved setup (§4.2.2) and MOTION. Our best-in-class memory consumption for massively parallel circuits, which is SEEC with dynamic layers (§4.1), interleaved setup (§4.2.2), and early freeing of SIMD gate outputs (§4.2.4), is particularly relevant in the outsourcing scenario [51]. SEEC's optimized memory consumption allows such a deployment to operate at a reduced cost, as servers need to utilize less memory.

5.1.4 Memory Impact of the Setup Phase (Fig. 5). We show the impact of precomputing MTs via oblivious transfer (OT) (Appendix A.3.3) on the peak memory consumption. MOTION's and SEEC's (without stored MTs as explained in §4.2.2) memory consumption increases the most due to the setup, whereas the increase in memory consumption of MP-SPDZ and ABY is smaller. The peak memory consumption of MP-SPDZ can be influenced by selecting the batch

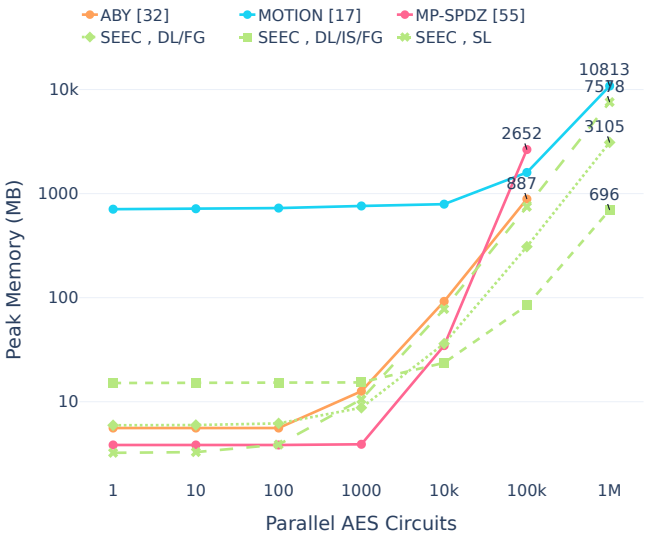


Figure 4: Peak memory consumption of parallel AES circuit evaluations with differing SIMD sizes N_P in the LAN setting excluding OT-based setup. $N_P = 10^6$ lead to errors for ABY and MP-SPDZ.

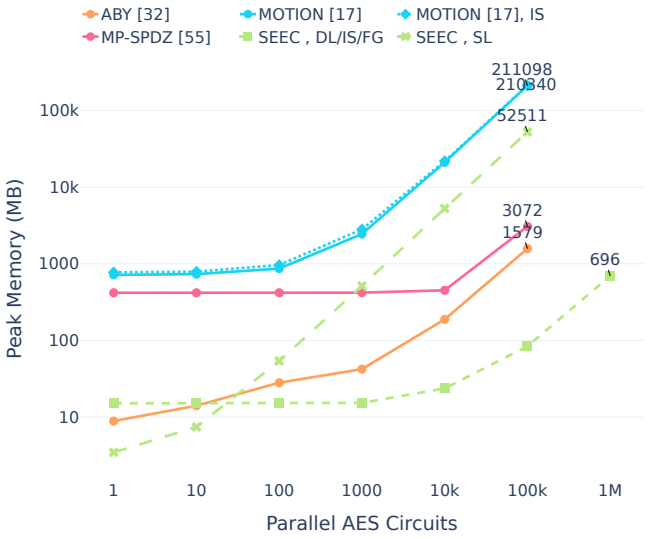


Figure 5: Peak memory consumption of parallel AES evaluations in the LAN setting including OT-based setup. MOTION, IS uses an interleaved setup.

size for OTs. SEEC with memory-reducing optimizations (§4.2), particularly the on-demand streaming of precomputed MTs (§4.2.2), provides the best memory efficiency. Our evaluation shows the impact of the setup phase and storage of MTs on the memory consumption of secret sharing-based MPC frameworks.

5.2 Runtime

As SEEC should achieve memory efficiency (see §5.1.1) and safety without compromizing on runtime, we investigate the effect of sub-circuits and optimizations (§4.2) on the online runtime of SEEC. Setup runtimes mostly depend on the circuit-depth-independent and hence constant-round OT implementation, which is not the focus of our work.

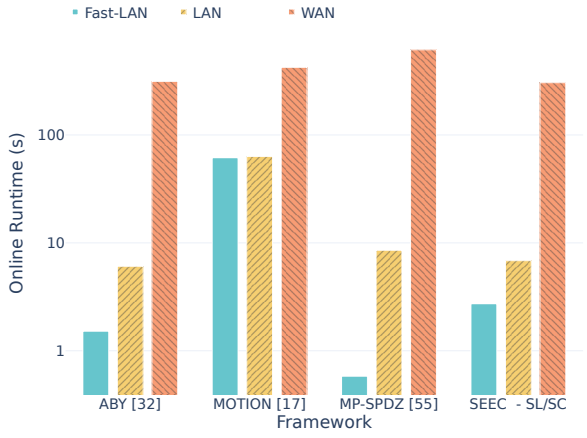


Figure 6: Online runtimes for $N_S = 100$ sequential AES circuits run on 32 cores in different network settings.

5.2.1 Online runtime of deep circuits (Figs. 6 and 7). We show the impact of the network setting on the online runtime of the evaluated frameworks in Fig. 6. MP-SPDZ is the slowest framework in the WAN setting and the fastest in the Fast-LAN setting. SEEC performs the best in the WAN setting. In the LAN setting, SEEC achieves a similar online runtime performance as MP-SPDZ for a single circuit with sequential AES evaluations. While ABY [31] is the fastest for every circuit size, MOTION [17] has the highest runtime overhead due to its asynchronous evaluation. MOTION's asynchronous evaluation mode adds considerable overhead when operating on such circuits that especially impacts the LAN setting.

For different SEEC variants, we observe that the dynamic layer iteration (§4.1) has a notable impact on the runtime. Precomputing the static layer representation (§4.2.1) yields the same runtime as without sub-circuits. In the high-latency WAN setting (Fig. 7b), the relative overhead of MOTION's asynchronous circuit evaluation is lower than in lower latency settings.

5.2.2 Efficiency across protocols (Table 2). To compare the efficiency of the different protocols ABY2.0 [81] and GMW [39] when using sub-circuits, we conduct runtime and communication benchmarks in the LAN setting. We report the results in Table 2. The main improvement of ABY2.0 over GMW is that the online communication cost is halved due to function-dependent preprocessing on paper. This effect is also reflected in our benchmarks, where ABY2.0 improves online communication 1.53× for SHA-256 and 1.86−1.96× for AES-CBC. The runtimes of ABY2.0 are slightly worse than for GMW in the online. This is due to more and slightly more expensive operations that need to be performed over ABY2.0 shares. We have also run SEEC using ABY2.0 and GMW using different Bristol circuits [2], for which we report the online communication, runtime, and peak memory in Appendix B.3.

Table 2: Online runtime in ms and communication of SEEC with static layer iteration (SL) for ABY2.0 [81] compared to GMW with differing numbers of sub-circuits.

Circuit	N_S	Protocol	Online Runtime (ms)	Online Communication
SHA-256	1	ABY2.0	**1,646**	**32.9 kB**
		GMW	1,676.92	50.3 kB
AES-CBC	1	ABY2.0	**71**	**1.9 kB**
		GMW	73	3.7 kB
	100	ABY2.0	7,534	**190 kB**
		GMW	**7,187**	355 kB
	10000	ABY2.0	850,662	**19 MB**
		GMW	**831,173**	35.5 MB

6 FURTHER RELATED WORK

Beyond ABY, MP-SPDZ, and MOTION (§2), we review further related work on efficient MPC implementations next. We discuss memory optimization advancements for GCs and compare the different implementation challenges to secret sharing-based protocols (§6.1). Afterward, we summarize MPC frameworks and compilers that translate high-level programs to efficient protocols (§6.2).

Framework	N_S Optim.	1	100	10,000
ABY [31]		**0.062**	**6.684**	**658.523**
MOTION [17]		0.187	18.712	–
MP-SPDZ [54]		0.087	8.527	849.446
SEEC	DL	0.081	8.325	830.017
	DL/SC (§4.1)	0.087	8.839	1,117.498
	SL/SC (§4.2.1)	0.073	7.187	831.173

(a) LAN setting

N_S Framework	1	10	100
ABY [31]	3.250	31.508	313.833
MOTION [17]	4.050	41.668	423.433
MP-SPDZ [54]	6.370	61.798	616.090
SEEC - SL/SC (§4.2.1)	**3.054**	**30.492**	**305.193**

(b) WAN setting

Figure 7: Online runtimes in seconds for N_S sequential AES circuits. Best results in bold.

6.1 Memory-Efficient Garbled Circuits

Garbled Circuits (GC), first introduced by Yao [98] allow secure two-party computation in a constant number of rounds. Here, the major challenge for memory and network consumption is the fact that the garbler sends a huge data packet for the GC to the evaluator. Several optimizations have reduced the size of the garbled tables [8, 10, 62, 79, 90, 100], and today's best construction takes 192 bits per AND gate [90].

6.1.1 Compact Circuit Representation. There is substantial prior work on the memory consumption of representing and executing GCs. [78] focuses on generating memory-efficient GCs to deploy secure function evaluation on mobile devices. They present the Pseudo Assembly Language (PAL) which acts as an intermediate representation (IR) between the high-level Secure Function Definition Language (SFDL) and low-level Secure Hardware Definition Language (SHDL) of the first MPC implementation Fairplay [74]. The compiler for the Portable Circuit Format (PCF) [64] improves on this with a compact bytecode IR that is optimized and executed by an interpreter. This allows the interpreter to perform online loop unrolling such that it is no longer required to hold in memory all garbled gates for every iteration. Similarly, [42] implements sub-circuits for GC with a focus on memory efficiency. Memory efficiency is also important for implementing GCs in hardware which was explored in [48, 49]. TinyGarble [93] proposes the use of logic synthesis tools to produce compact sequential circuits. Sequential circuits are more expressive than combinational circuits, which are directed acyclic graphs that are often used in MPC implementations. Sequential circuits support clock cycles and states that may change with each cycle which allows a compact loop representation. For example, a 1024-bit addition circuit can be described as a single 1-bit full adder that is evaluated for 1024 sequential cycles. The swanky [37] Rust library is a suite of MPC and zero-knowledge proof (ZKP) protocols, including oblivious transfer, private set intersection, and other secure computation utilities like serialization and communication channel abstractions. While swanky implements GCs, it does not contain secret sharing-based protocols like GMW. This is unlike SEEC which is rather built specifically with the specialties of secret sharing-based protocols in mind.

6.1.2 Pipelining. To reduce memory consumption of GCs, pipelining the GC generation and evaluation is effective, as demonstrated by [48, 73] and was used in subsequent GC implementations [6, 42, 44, 65, 70]. Instead of garbling and sending the whole circuit, garbling and evaluation are performed in a pipelining fashion. Thus, only a subset of the GC is stored in memory which significantly reduces memory consumption. However, this close coupling of a pipelined setup and online phase can slow down the online phase.

Since GC protocols are constant-round, there is no penalty for executing sub-functions sequentially. However, for secret sharing-based MPC protocols, independent sub-functions must be evaluated in parallel to reduce the round complexity and latency. Pipelined evaluation for generation of multiplication triples in secret sharing-based MPC protocols was implemented in [17, 54]. However, pipelining gets more complicated with function-dependent preprocessing, as in [81]. Here, the batching of preprocessing material, which now depends on the function, must be coordinated with the interactive multi-round online phase.

6.2 MPC Frameworks and Compilers

There are many MPC frameworks and compilers. In the context of MPC, frameworks frequently incorporate compilers and the distinction between a framework and a compiler can get blurry.

6.2.1 MPC Frameworks for Secret Sharing. Sharemind [12, 13] is one of the earliest frameworks for secret sharing-based protocols. It compiles a domain-specific language (DSL) for evaluation with a three-party protocol. Wysteria [84, 85] is a high-level DSL that supports mixed-protocol evaluation. Its interpreter produces a circuit representation with function call inlining which is evaluated by the GMW implementation of [25]. FRESCO [3] is a Java framework focusing on developer usability and implements SPDZ-style secret sharing-protocols [27, 29, 83]. SCALE-MAMBA [66] and MP-SPDZ [54] implement numerous secret sharing-based protocols. SCALE-MAMBA solely implements maliciously secure protocols and fewer optimization passes compared to MP-SPDZ. SCALE-MAMBA is written in a mix of C/C++ and Rust. However, the Rust part mainly considers compiling for the VM, while the runtime is mostly written in C/C++. We discuss MP-SPDZ in more detail in §2.3. FLUTE [19] is a recent work on fast lookup table evaluations, generalizing boolean gates in ABY2.0 [81]. The main contributions of FLUTE are protocol improvements with prototypical Rust implementations. The Rust implementation contains the LUT protocols with ABY2.0 sharing semantics and Boolean GMW for benchmarks. In contrast, SEEC implements the full secret sharing-based ABY2.0 protocol family, including Boolean, Arithmetic (including vector and matrix multiplications), mixed-mode

ABY2.0 and Boolean, Arithmetic, mixed-mode GMW. Focusing on usability, MPyC [92] implements threshold secret sharing over finite fields and utilizes the co-routine support in Python to define a comprehensive runtime and operations library with asynchronous evaluation. [28] and [72] also investigated asynchronous evaluation for MPC.

6.2.2 MPC Frameworks for Garbled Circuits. The first implementation of GCs and MPC in general was Fairplay [74] which contains a compiler that translates programs written in SFDL into a circuit representation called SHDL that is evaluated with Yao's GC protocol [98]. FairplayMP [11] extends this to the multi-party setting by implementing an improved version of the BMR protocol [8]. CBMC-GC [43] compiles ANSI-C programs to boolean circuits for evaluation with GCs. Similarly, Frigate [77] compiles a C-like language to boolean circuits with any number of inputs. [64] presents a compiler that translates bytecode generated from a C compiler into a compact bytecode called Portable Circuit Format (PCF). The PCF code is evaluated by an interpreter that constructs a circuit on the fly. This idea is similar to our sub-circuits, where the evaluation layers are lazily computed using iterators (§4.1). Obliv-C [99] is a transpiler for an extension of C to plain C code for secure computation. TinyGarble [93] and subsequent works [45] employ industry-grade logic synthesis tools to generate compact circuit representations in the form of sequential circuits which was extended to secret sharing-based protocols by [30]. [78] focuses on memory-efficient GCs to enable deployment on memory-constrained devices. [65] were able to evaluate a boolean circuit with billions of gates with a maliciously secure GC protocol by re-generating the circuit instead of keeping it in memory the whole time.

6.2.3 Frameworks for Mixed-Protocol MPC. TASTY [41] is the first work for mixed-protocol MPC and mixes GC with Homomorphic Encryption and compiles from a subset of Python. HyCC [21] compiles ANSI-C into an optimized mixed-protocol MPC representation. For this, the HyCC decomposes a program into multiple modules and compiles each to boolean and arithmetic circuits. The circuits are independently optimized and can be evaluated by MPC frameworks such as ABY [31] or MOTION [16, 17]. Because ABY does not support sub-circuits, any calls in the HyCC representation are completely inlined into one circuit. This also holds for the ABY backends in EzPC [24] and CrypTFlow [67, 86]. OPA [47], Silph [103], and CostCO [35] provide optimized protocol assignment and scheduling for mixed-protocol evaluation. CostCO [35] is a Python framework that models the cost of MPC operations. It runs microbenchmarks of different operations under different MPC protocols to estimate their cost when composing them in bigger secure programs. CostCO uses ABY [31] and agmpc [96] internally. The generated cost models are used by the Silph compiler framework [103]. Silph uses the cost models to efficiently compile a given program into a mixed circuit with optimal protocol mixing of the ABY protocol family [31]. For secure evaluation, Silph uses ABY as its MPC backend.

6.2.4 MPC Compilers. Sub-circuits have been explored for efficient MPC circuit compilation. [20] provide an extension of CBMC-GC [43] which splits the original source code into multiple sub-circuits, runs optimizations on each sub-circuit and merges them again together into one global circuit for evaluation. The IRs of HyCC [21], FUSE [18], and Silph [103] support function calls on the IR level and are designed for the mixed-protocol setting including secret sharing-based protocols. However, when evaluating any IR with ABY [31], MP-SPDZ [54], and MOTION [17], the function calls are inlined inside the frameworks' in-memory representations. This is not the case for SEEC, as it maintains the sub-circuits from FUSE IR and uses this during evaluation for low memory consumption. FUSE [18] is a compiler framework with several optimization and analysis passes, including automatic SIMD vectorization and subcircuit replacement. FUSE IR is both a flexible in-memory and file format based on flatbuffers [40], enabling cross-platform usage.

In the context of efficient secret sharing-based MPC, several compiler works have studied the problem of compiling circuits with low multiplicative depth [22, 30, 82]. Demmler et al. [30] and SynCirc [82] use hardware synthesis tools to achieve this while ShallowCC [22] provide a compiler extension to CBMC-GC [43] that compiles ANSI-C to low depth Boolean circuits.

SIMD gates significantly improve efficiency as shown in [91], so they are implemented in most modern MPC frameworks [9, 16, 17, 31, 38, 54, 58, 101]. [69] propose a compiler framework for automatic SIMD vectorization. Their IR does not support mixed-protocol settings or sub-circuits. GraphSC [80] presents programming abstractions for automatic parallelization. Expression localization reduces the expressions that are computed with MPC inside a program. This notion was first presented in [44] and automated by Kerschbaum [59, 60].

7 CONCLUSION AND FUTURE WORK

We envision multiple future directions for SEEC. As we focused on improving secret-sharing-based protocol implementations, we have left out GC protocols. To utilize the complete set of A/B/Y protocols for applications, SEEC can be extended with a pipelined evaluation of the three-halves GC protocol [90]. Our evaluation showed that layer-by-layer gate evaluation performs worse than asynchronous evaluation for very large SIMD sizes. To tackle this, adding an asynchronous version of the `Executor` with stackless futures allows the utilization of both evaluation variants, depending on the use case. The asynchronous evaluation also paves the way to efficiently evaluate gates with differing communication rounds or computation complexities, necessary for the mixing of garbled circuits and secret-sharing-based protocols.

ACKNOWLEDGMENTS

The authors thank the shepherd and the anonymous reviewers for their helpful inputs and support for this paper. We also thank Raine Nieminen for his initial inputs that led to this project. The authors used Grammarly to revise the text in the paper.

This project received funding from the European Research Council (ERC) under the European Union's research and innovation programs Horizon 2020 (PSOTI/850990) and Horizon Europe (PRIVTOOLS/101124778). It was co-funded by the Deutsche Forschungsgemeinschaft (DFG) within SFB 1119 CROSSING/236615297 and GRK 2050 Privacy & Trust/251805230.

REFERENCES

[1] 2020. MPC Alliance. https://www.mpcalliance.org.

[2] Victor Arribas Abril, Pieter Maene, Nele Mertens, Danilo Sijacic, and Nigel Smart. 2019. Bristol Fashion MPC Circuits. https://nigelsmart.github.io/MPC-Circuits/.

[3] Alexandra Institute. 2022. FRESCO: A FRamework for Efficient Secure COmputation. https://github.com/aicis/fresco.

[4] David W. Archer, Dan Bogdanov, Yehuda Lindell, Liina Kamm, Kurt Nielsen, Jakob Illeborg Pagter, Nigel P. Smart, and Rebecca N. Wright. 2018. From Keys to Databases - Real-World Applications of Secure Multi-Party Computation. *Comput. J.* (2018).

[5] Gilad Asharov, Yehuda Lindell, Thomas Schneider, and Michael Zohner. 2013. More Efficient Oblivious Transfer and Extensions for Faster Secure Computation. In *CCS*.

[6] Marshall Ball, Brent Carmer, Tal Malkin, Mike Rosulek, and Nichole Schimanski. 2019. Garbled Neural Networks are Practical. Cryptology ePrint Archive, Report 2019/338.

[7] Donald Beaver. 1992. Efficient multiparty protocols using circuit randomization. In *CRYPTO*.

[8] Donald Beaver, Silvio Micali, and Phillip Rogaway. 1990. The Round Complexity of Secure Protocols (Extended Abstract). In *STOC*.

[9] Gabrielle Beck, Aarushi Goel, Abhishek Jain, and Gabriel Kaptchuk. 2021. Order-C secure multiparty computation for highly repetitive circuits. In *EUROCRYPT*.

[10] Mihir Bellare, Viet Tung Hoang, Sriram Keelveedhi, and Phillip Rogaway. 2013. Efficient Garbling from a Fixed-Key Blockcipher. In *S&P*.

[11] Assaf Ben-David, Noam Nisan, and Benny Pinkas. 2008. FairplayMP: A System for Secure Multi-Party Computation. In *CCS*.

[12] Dan Bogdanov, Roman Jagomägis, and Sven Laur. 2012. A universal toolkit for cryptographically secure privacy-preserving data mining. In *PAISI*.

[13] Dan Bogdanov, Sven Laur, and Jan Willemson. 2008. Sharemind: A framework for fast privacy-preserving computations. In *ESORICS*.

[14] Dan Bogdanov, Riivo Talviste, and Jan Willemson. 2012. Deploying Secure Multi-Party Computation for Financial Data Analysis - (Short Paper). In *FC*.

[15] Elette Boyle, Geoffroy Couteau, Niv Gilboa, Yuval Ishai, Lisa Kohl, and Peter Scholl. 2019. Efficient Pseudorandom Correlation Generators: Silent OT Extension and More. In *CRYPTO*.

[16] Lennart Braun, Rosario Cammarota, and Thomas Schneider. 2021. POSTER: A Generic Hybrid 2PC Framework with Application to Private Inference of Unmodified Neural Networks. *Privacy in Machine Learning Workshop (PriML)* (2021).

[17] Lennart Braun, Daniel Demmler, Thomas Schneider, and Oleksandr Tkachenko. 2022. MOTION - A Framework for Mixed-Protocol Multi-Party Computation. *TOPS* (2022).

[18] Lennart Braun, Moritz Huppert, Nora Khayata, Thomas Schneider, and Oleksandr Tkachenko. 2023. FUSE - Flexible File Format and Intermediate Representation for Secure Multi-Party Computation. In *ASIACCS*.

[19] Andreas Brüggemann, Robin Hundt, Thomas Schneider, Ajith Suresh, and Hossein Yalame. 2023. FLUTE: Fast and Secure Lookup Table Evaluations. In *S&P*.

[20] Niklas Buescher, David Kretzmer, Arnav Jindal, and Stefan Katzenbeisser. 2016. Scalable secure computation from ANSI-C. In *International Workshop on Information Forensics and Security (WIFS)*.

[21] Niklas Büscher, Daniel Demmler, Stefan Katzenbeisser, David Kretzmer, and Thomas Schneider. 2018. HyCC: Compilation of Hybrid Protocols for Practical Secure Computation. In *CCS*.

[22] Niklas Büscher, Andreas Holzer, Alina Weber, and Stefan Katzenbeisser. 2016. Compiling low depth circuits for practical secure computation. In *ESORICS*.

[23] Henry Carter, Benjamin Mood, Patrick Traynor, and Kevin Butler. 2015. Outsourcing secure two-party computation as a black box. In *CANS*.

[24] Nishanth Chandran, Divya Gupta, Aseem Rastogi, Rahul Sharma, and Shardul Tripathi. 2019. EzPC: Programmable and Efficient Secure Two-Party Computation for Machine Learning. In *EuroS&P*.

[25] Seung Geol Choi, Kyung-Wook Hwang, Jonathan Katz, Tal Malkin, and Dan Rubenstein. 2012. Secure Multi-Party Computation of Boolean Circuits with Applications to Privacy in On-Line Marketplaces. In *CT-RSA*. Springer.

[26] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. 2022. *Introduction to Algorithms, 4th Edition.* MIT Press.

[27] Ronald Cramer, Ivan Damgård, Daniel Escudero, Peter Scholl, and Chaoping Xing. 2018. SPD$\mathbb{Z}_{2^k}$: Efficient MPC mod 2^k for Dishonest Majority. In *CRYPTO*.

[28] Ivan Damgård, Martin Geisler, Mikkel Krøigaard, and Jesper Buus Nielsen. 2009. Asynchronous multiparty computation: Theory and implementation. In *PKC*.

[29] Ivan Damgård, Valerio Pastro, Nigel Smart, and Sarah Zakarias. 2012. Multiparty computation from somewhat homomorphic encryption. In *CRYPTO*.

[30] Daniel Demmler, Ghada Dessouky, Farinaz Koushanfar, Ahmad-Reza Sadeghi, Thomas Schneider, and Shaza Zeitouni. 2015. Automated synthesis of optimized circuits for secure computation. In *CCS*.

[31] Daniel Demmler, Thomas Schneider, and Michael Zohner. 2015. ABY – A Framework for Efficient Mixed-Protocol Secure Two-Party Computation. In *NDSS*.

[32] Henri Dohmen, Robin Hundt, Nora Khayata, and Thomas Schneider. 2025. SEEC: Memory Safety Meets Efficiency in Secure Two-Party Computation. In *ASIACCS*. https://encrypto.de/code/SEEC

[33] Kasra EdalatNejad, Wouter Lueks, Julien Pierre Martin, Soline Ledésert, Anne L'Hôte, Bruno Thomas, Laurent Girod, and Carmela Troncoso. 2020. DatashareNetwork: A Decentralized Privacy-Preserving Search Engine for Investigative Journalists. In *USENIX Security*.

[34] Steven Englehardt. 2019. Next steps in privacy-preserving telemetry with Prio. https://blog.mozilla.org/security/2019/06/06/next-steps-in-privacy-preserving-telemetry-with-prio/.

[35] Vivian Fang, Lloyd Brown, William Lin, Wenting Zheng, Aurojit Panda, and Raluca Ada Popa. 2022. CostCO: An Automatic Cost Modeling Framework for Secure Multi-Party Computation. In *EuroS&P*.

[36] Jun Furukawa, Yehuda Lindell, Ariel Nof, and Or Weinstein. 2017. High-throughput secure three-party computation for malicious adversaries and an honest majority. In *EUROCRYPT*.

[37] Galois, Inc. 2019. swanky: A suite of Rust libraries for secure computation. https://github.com/GaloisInc/swanky.

[38] Daniel Genkin, Yuval Ishai, and Antigoni Polychroniadou. 2015. Efficient multi-party computation: From passive to active security via secure SIMD circuits. In *CRYPTO*.

[39] Oded Goldreich, Silvio Micali, and Avi Wigderson. 1987. How to Play any Mental Game or A Completeness Theorem for Protocols with Honest Majority. In *STOC*.

[40] Google. 2014. FlatBuffers. https://github.com/google/flatbuffers.

[41] Wilko Henecka, Stefan Kögl, Ahmad-Reza Sadeghi, Thomas Schneider, and Immo Wehrenberg. 2010. TASTY: Tool for Automating Secure Two-Party Computations. In *CCS*.

[42] Wilko Henecka and Thomas Schneider. 2013. Faster Secure Two-Party Computation with Less Memory. In *ASIACCS*.

[43] Andreas Holzer, Martin Franz, Stefan Katzenbeisser, and Helmut Veith. 2012. Secure two-party computations in ANSI C. In *CCS*.

[44] Yan Huang, David Evans, Jonathan Katz, and Lior Malka. 2011. Faster Secure Two-Party Computation Using Garbled Circuits. In *USENIX Security*.

[45] Siam U. Hussain, Baiyu Li, Farinaz Koushanfar, and Rosario Cammarota. 2020. TinyGarble2: Smart, Efficient, and Scalable Yao's Garble Circuit. In *PPMLP@CCS*.

[46] Y. Ishai, J. Kilian, K. Nissim, and E. Petrank. 2003. Extending Oblivious Transfers Efficiently. In *CRYPTO*.

[47] Muhammad Ishaq, Ana L. Milanova, and Vassilis Zikas. 2019. Efficient MPC via Program Analysis: A Framework for Efficient Optimal Mixing. In *CCS*.

[48] Kimmo Järvinen, Vladimir Kolesnikov, Ahmad-Reza Sadeghi, and Thomas Schneider. 2010. Embedded SFE: Offloading server and network using hardware tokens. In *FC*.

[49] Kimmo Järvinen, Vladimir Kolesnikov, Ahmad-Reza Sadeghi, and Thomas Schneider. 2010. Garbled Circuits for Leakage-Resilience: Hardware Implementation and Evaluation of One-Time Programs. In *CHES*.

[50] Daniel Kales, Christian Rechberger, Thomas Schneider, Matthias Senker, and Christian Weinert. 2019. Mobile private contact discovery at scale. In *USENIX Security*.

[51] Seny Kamara, Payman Mohassel, and Mariana Raykova. 2011. Outsourcing Multi-Party Computation. Cryptology ePrint Archive, Paper 2011/272.

[52] Jonathan Katz and Yehuda Lindell. 2007. *Introduction to modern cryptography: Principles and protocols.* Chapman and hall/CRC.

[53] Darya Kaviani and Raluca Ada Popa. 2023. MPC Deployments. https://mpc.cs.berkeley.edu.

[54] Marcel Keller. 2020. MP-SPDZ: A Versatile Framework for Multi-Party Computation. In *CCS*.

[55] Marcel Keller. 2024. How To Scale Multi-Party Computation. Cryptology ePrint Archive, Paper 2024/1990.

[56] Marcel Keller, Emmanuela Orsini, and Peter Scholl. 2016. MASCOT: faster malicious arithmetic secure computation with oblivious transfer. In *CCS*.

[57] Marcel Keller, Dragos Rotaru, Peter Scholl, and Nigel P. Smart. 2018. SPDZ-2. https://github.com/bristolcrypto/SPDZ-2.

[58] Marcel Keller, Peter Scholl, and Nigel P Smart. 2013. An architecture for practical actively secure MPC with dishonest majority. In *CCS*.

[59] Florian Kerschbaum. 2011. Automatically optimizing secure computation. In *CCS*.

[60] Florian Kerschbaum. 2013. Expression rewriting for optimizing secure computation. In *CODASPY*.

[61] Brian Knott, Shobha Venkataraman, Awni Y. Hannun, Shubho Sengupta, Mark Ibrahim, and Laurens van der Maaten. 2021. CrypTen: Secure Multi-Party Computation Meets Machine Learning. In *NeurIPS*.

[62] Vladimir Kolesnikov and Thomas Schneider. 2008. Improved Garbled Circuit: Free XOR Gates and Applications. In *ICALP*.

[63] Nishat Koti, Mahak Pancholi, Arpita Patra, and Ajith Suresh. 2021. SWIFT: Super-fast and Robust Privacy-Preserving Machine Learning. In *USENIX Security*.

[64] Benjamin Kreuter, abhi shelat, Benjamin Mood, and Kevin R. B. Butler. 2013. PCF: A Portable Circuit Format for Scalable Two-Party Secure Computation. In *USENIX Security*.

[65] Benjamin Kreuter, abhi shelat, and Chih-Hao Shen. 2012. Billion-Gate Secure Computation with Malicious Adversaries. In *USENIX Security*.

[66] KULeuven-COSIC. 2019. SCALE-MAMBA. https://github.com/KULeuven-COSIC/SCALE-MAMBA.

[67] Nishant Kumar, Mayank Rathee, Nishanth Chandran, Divya Gupta, Aseem Rastogi, and Rahul Sharma. 2020. CrypTFlow: Secure TensorFlow Inference. In *S&P*.

[68] Adam Langley, Alistair Riddoch, Alyssa Wilk, Antonio Vicente, Charles Krasic, Dan Zhang, Fan Yang, Fedor Kouranov, Ian Swett, Janardhan Iyengar, et al. 2017. The QUIC transport protocol: Design and internet-scale deployment. In *SIGCOMM*.

[69] Benjamin Levy, Ben Sherman, Muhammad Ishaq, Lindsey Kennard, Ana L. Milanova, and Vassilis Zikas. 2023. Compilation and Backend-Independent Vectorization for Multi-Party Computation. In *CCS*.

[70] Chang Liu, Xiao Shaun Wang, Kartik Nayak, Yan Huang, and Elaine Shi. 2015. ObliVM: A Programming Framework for Secure Computation. In *S&P*.

[71] Xiaoning Liu, Yifeng Zheng, Xingliang Yuan, and Xun Yi. 2022. Securely outsourcing neural network inference to the cloud with lightweight techniques. *IEEE Transactions on Dependable and Secure Computing* (2022).

[72] Donghang Lu, Thomas Yurek, Samarth Kulshreshtha, Rahul Govind, Aniket Kate, and Andrew Miller. 2019. Honeybadgermpc and asynchromix: Practical asynchronous mpc and its application to anonymous communication. In *CCS*.

[73] Lior Malka. 2011. VMCrypt: Modular Software Architecture for Scalable Secure Computation. In *CCS*.

[74] Dahlia Malkhi, Noam Nisan, Benny Pinkas, and Yaron Sella. 2004. Fairplay - Secure Two-Party Computation System. In *USENIX Security*.

[75] Silvio Micali, Oded Goldreich, and Avi Wigderson. 1987. How to play ANY mental game. In *STOC*.

[76] Payman Mohassel and Yupeng Zhang. 2017. SecureML: A system for scalable privacy-preserving machine learning. In *S&P*.

[77] Benjamin Mood, Debayan Gupta, Henry Carter, Kevin R. B. Butler, and Patrick Traynor. 2016. Frigate: A Validated, Extensible, and Efficient Compiler and Interpreter for Secure Computation. In *EuroS&P*.

[78] Benjamin Mood, Lara Letaw, and Kevin Butler. 2012. Memory-Efficient Garbled Circuit Generation for Mobile Devices. In *FC*.

[79] Moni Naor, Benny Pinkas, and Reuban Sumner. 1999. Privacy Preserving Auctions and Mechanism Design. In *Electronic Commerce*.

[80] Kartik Nayak, Xiao Shaun Wang, Stratis Ioannidis, Udi Weinsberg, Nina Taft, and Elaine Shi. 2015. GraphSC: Parallel secure computation made easy. In *S&P*.

[81] Arpita Patra, Thomas Schneider, Ajith Suresh, and Hossein Yalame. 2021. ABY2.0: Improved Mixed-Protocol Secure Two-Party Computation. In *USENIX Security*.

[82] Arpita Patra, Thomas Schneider, Ajith Suresh, and Hossein Yalame. 2021. SynCirc: Efficient synthesis of depth-optimized circuits for secure computation. In *HOST*.

[83] Samuel Ranellucci. 2017. The TinyTable Protocol for 2-Party Secure Computation, or: Gate-Scrambling Revisited. In *CRYPTO*.

[84] Aseem Rastogi, Matthew A. Hammer, and Michael Hicks. 2014. Wysteria: A Programming Language for Generic, Mixed-Mode Multiparty Computations. In *S&P*.

[85] Aseem Rastogi, Nikhil Swamy, and Michael Hicks. 2017. WYS*: A DSL for Verified Secure Multi-party Computations. *arXiv preprint arXiv:1711.06467* (2017).

[86] Deevashwer Rathee, Mayank Rathee, Nishant Kumar, Nishanth Chandran, Divya Gupta, Aseem Rastogi, and Rahul Sharma. 2020. CrypTFlow2: Practical 2-Party Secure Inference. In *CCS*.

[87] Deevashwer Rathee, Thomas Schneider, and KK Shukla. 2019. Improved multiplication triple generation over rings via RLWE-based AHE. In *CANS*.

[88] C++ Reference. 2023. Zero-overhead principle. https://en.cppreference.com/w/cpp/language/Zero-overhead_principle.

[89] M. Sadegh Riazi, Christian Weinert, Oleksandr Tkachenko, Ebrahim M. Songhori, Thomas Schneider, and Farinaz Koushanfar. 2018. Chameleon: A Hybrid Secure Computation Framework for Machine Learning Applications. In *ASIACCS*.

[90] Mike Rosulek and Lawrence Roy. 2021. Three Halves Make a Whole? Beating the Half-Gates Lower Bound for Garbled Circuits. In *CRYPTO*.

[91] Thomas Schneider and Michael Zohner. 2013. GMW vs. Yao? Efficient Secure Two-Party Computation with Low Depth Circuits. In *FC*.

[92] Berry Schoenmakers. 2018. MPyC—Python Package for Secure Multiparty Computation. In *Workshop on the Theory and Practice of MPC*. https://github.com/lschoe/mpyc

[93] Ebrahim M. Songhori, Siam U. Hussain, Ahmad-Reza Sadeghi, Thomas Schneider, and Farinaz Koushanfar. 2015. TinyGarble: Highly Compressed and Scalable Sequential Garbled Circuits. In *S&P*.

[94] Bjarne Stroustrup. 1994. *The Design and Evolution of C++*. Addison Wesley.

[95] Aaron Turon. 2015. Abstraction without overhead: traits in Rust. https://blog.rust-lang.org/2015/05/11/traits.html.

[96] Xiao Wang, Samuel Ranellucci, and Jonathan Katz. 2017. Global-Scale Secure Multiparty Computation. CCS.

[97] Andrew Chi-Chih Yao. 1982. Protocols for Secure Computations (Extended Abstract). In *FOCS*.

[98] Andrew Chi-Chih Yao. 1986. How to Generate and Exchange Secrets (Extended Abstract). In *FOCS*.

[99] Samee Zahur and David Evans. 2015. Obliv-C: A Language for Extensible Data-Oblivious Computation. Cryptology ePrint Archive, Report 2015/1153.

[100] Samee Zahur, Mike Rosulek, and David Evans. 2015. Two Halves Make a Whole - Reducing Data Transfer in Garbled Circuits Using Half Gates. In *EUROCRYPT*.

[101] Wenting Zheng, Ryan Deng, Weikeng Chen, Raluca Ada Popa, Aurojit Panda, and Ion Stoica. 2021. CEREBRO: A platform for Multi-Party cryptographic collaborative learning. In *USENIX Security*.

[102] Yifeng Zheng, Huayi Duan, and Cong Wang. 2019. Towards secure and efficient outsourcing of machine learning classification. In *ESORICS*.

[103] Jinhao Zhu, Alex Ozdemir, Riad Wahby, Fraser Brown, and Wenting Zheng. 2022. Silph: A Framework for Scalable and Accurate Generation of Hybrid MPC Protocols. In *S&P*.

A PRELIMINARIES

Let $\mathcal{P}_0$ and $\mathcal{P}_1$ denote two parties that carry out secure computation over the ring $\mathbb{Z}_{2^k}$, where $k \in \mathbb{N} \setminus \{0\}$. In practice, these two parties can be instantiated by two non-colluding servers for computation-heavy tasks like privacy-preserving machine learning in the outsourcing scenario [24, 51]. For other services like privacy-preserving contact discovery [50], one party is a server and the other is the client. Our framework can be used in both settings.

Secure Multi-Party Computation (MPC) allows $\mathcal{P}_0$ and $\mathcal{P}_1$ to jointly compute a function on their private inputs, such that no information beyond the output of the function is leaked. In an ideal world, $\mathcal{P}_0$ and $\mathcal{P}_1$ would send their inputs to a trusted party that computes the function and returns the output. MPC protocols allow $\mathcal{P}_0$ and $\mathcal{P}_1$ to securely simulate this trusted party.

A.1 Circuits

MPC protocols are commonly defined to operate on a circuit representation of the function to compute. This circuit is commonly represented as a DAG where the set of nodes V are the gates the protocol supports, e.g. XOR and AND for GMW [39]. Edges between nodes correspond to wires connecting gates. We define a base circuit C_i as a tuple $(i, V_i, E_i, I_i, O_i, \phi)$ consisting of the circuit identifier $i \in \mathbb{N}$, the set of gates $G_i \subset \mathbb{N}$, the wires between gates $W_i \subset \{(x, y) \mid (x, y) \in G_i^2\}$, the input gates $I_i \subset V_i$, the output gates $O_i \subset V_i$, and the mapping function $\phi : V_i \to \mathcal{G}$ that maps a gate identifier to its corresponding gate type. For a Boolean circuit, $\mathcal{G}$ is set to $\{\text{AND}, \text{XOR}, \text{NOT}\}$. The graph defined by G_i and W_i must be acyclic and there is no gate which is contained in I_i and O_i.

A.2 Adversary Model

The adversary $\mathcal{A}$ corrupts a subset of the parties. Thus, in the two-party setting, $\mathcal{A}$ corrupts either $\mathcal{P}_0$ or $\mathcal{P}_1$. We differentiate between two types of adversaries: (1) The semi-honest, or passive, adversary is honest-but-curious, i.e., it follows the protocol without any deviations but tries to learn additional information about the inputs. (2) The malicious, or active, adversary may arbitrarily deviate from the protocol, but cheating is detected with an overwhelming probability. In this work, we focus on semi-honest adversaries because they are more efficient in terms of computation and communication complexity and often suffice in practice [34]. However, the design of our framework SEEC is not limited to semi-honest security and can be extended with additional checks to provide maliciously secure secret sharing-based protocols in the style of SPDZ [29, 57].

A.3 Secret Sharing-based Protocols

The idea of secret sharing in the two-party setting is that a secret value x is split into two additive shares x_0 and x_1 such that $x_0 + x_1 = x$. Here, $\mathcal{P}_0$ holds share x_i and each share x_0 and x_1 "looks random", i.e., seeing only one share provides no information about the original secret (see [52]).

We revisit two protocol variants of semi-honest secret sharing-based MPC: GMW [75] and ABY2.0 [81]. We assume that computation is done over the ring $\mathbb{Z}_{2^k}$, where operations are performed modulo 2^k. In the special case of $k = 1$, addition corresponds to XOR (exclusive-or) and multiplication to AND.

A.3.1 GMW [75].

We revisit the GMW protocol from [75] with Beaver's multiplication triples (MTs) [7, 91].

Sharing Semantics. The $[\cdot]$-sharing of a value $x \in \mathbb{Z}_{2^k}$ is an additively homomorphic sharing of x, i.e., $\mathcal{P}_0$ holds $[x]_0$ and $\mathcal{P}_1$ holds $[x]_1$ such that $[x]_0 + [x]_1 = x$.

To compute a secret share, $\mathcal{P}_i$ who holds the secret x, samples $[x]_0 \leftarrow_\$ \mathbb{Z}_{2^k}$, sets $[x]_1 = x - [x]_1$ and sends $[x]_{1-i}$ to $\mathcal{P}_{1-i}$, where $i \in \{0, 1\}$. To publicly reconstruct a value x from the secret shares, $\mathcal{P}_i$ sends $[x]_i$ to $\mathcal{P}_{1-i}$ and both parties compute $x = [x]_0 + [x]_1$.

Linear Operations. Given two secret shares $[x]$, $[y]$ and constants α, β, γ, the parties can non-interactively compute linear combinations of $[x]$ and $[y]$ by computing $\alpha[x]_i + \beta[y]_i + i\gamma$.

Multiplication. The original multiplication protocol of [75] utilizes Oblivious Transfer (OT), but [7] shows how to more efficiently realize multiplication with precomputed multiplication triples (MTs). An MT is a triple $([a]_i, [b]_i, [c]_i)$ such that $(\sum_i [a]_i) \cdot (\sum_i [b]_i) = \sum_i [c]_i$, for $i \in \{0, 1\}$, i.e., $a \cdot b = c$. Given the shares of a precomputed MT $([a]_i, [b]_i, [c_i])$, the parties interactively compute the multiplication of two secret shares $[x]$ and $[y]$ as follows. $\mathcal{P}_i$ locally computes $[e]_i := [x]_i + [a]_i$ and $[d]_i := [y]_i + [b]_i$ and reconstructs e and d by sending the shares $[e]_i$, $[d]_i$ to $\mathcal{P}_{1-i}$. This requires sending two independent messages for each layer of MULT/AND gates. Given e, d, each party $\mathcal{P}_i$ computes the output share as $[xy] = ied + d[b]_i + e[a]_i + [c]_i$.

A.3.2 ABY2.0 [81].

We revisit the protocol from [81] which improves the online communication over GMW by 2× using function-dependent preprocessing.

Sharing Semantics. In addition to the $[\cdot]$-sharing, which is the same as in GMW (see Appendix A.3.1), the $\langle\cdot\rangle$-sharing of x is a tuple $([\delta_x], \Delta_x)$ such that $\Delta_x = x + \delta_x$. An individual $\langle\cdot\rangle$-share of a value x held by Party $\mathcal{P}_i$ is a tuple of its components $\langle x\rangle_i = ([\delta_x]_i, \Delta_x)$. Note that Δ_x is known to both parties.

To compute a $\langle\cdot\rangle$-share of $\mathcal{P}_i$'s secret value x, the party $\mathcal{P}_i$ samples $[\delta_x]_i \leftarrow_\$ \{0,1\}^k$ and both parties sample $[\delta_x]_{1-i} \leftarrow_\$ \{0,1\}^k$, e.g., by using pre-shared keys and a pseudo-random generator. $\mathcal{P}_i$ can compute the masking value $\delta_x := [\delta_x]_i + [\delta_x]_{1-i}$ and with that compute $\Delta_x = x + \delta_x$. Each party $\mathcal{P}_i$ then sets $\langle x\rangle_i = ([\delta_x]_i, \Delta_x)$. To reconstruct x from its $\langle\cdot\rangle$-sharing, $\mathcal{P}_i$ sends $[\delta_x]_i$ to $\mathcal{P}_{1-i}$. Both parties then locally compute $x = \Delta_x - [\delta_x]_0 - [\delta_x]_1$.

Linear Operations. Similar to GMW (see Appendix A.3.1), linear combinations of secret shares of x, y and constants α, β, γ can be computed non-interactively. For $[\cdot]$-shares, this is equal to GMW. For $\langle\cdot\rangle$-shares, the parties locally compute $(\alpha[\delta_x]_i + \beta[\delta_y]_i + \gamma, \alpha\Delta_x + \beta\Delta_y + \gamma)$.

Multiplication. Given two shares $\langle x\rangle$, $\langle y\rangle$ and intended output of the multiplication $z := xy$, we observe that $z = xy = (\delta_x + \Delta_x)(\delta_y + \Delta_y) = \delta_x\delta_y + \delta_x\Delta_y + \delta_y\Delta_x + \Delta_x\Delta_y$. Since Δ_x and Δ_y are known to both parties in the clear, everything apart from $\delta_x\delta_y$ are linear combinations and can thus be computed locally. The leftover term $\delta_x\delta_y$ is input-independent as the $[\cdot]$-shares of δ_x and δ_y were obtained by sampling random values. Hence, the $[\cdot]$-shares of $\delta_x\delta_y$ can be precomputed offline, i.e., when the inputs are not yet known (see Appendix A.3.3). Note that δ_{xy} is input-independent, but function-dependent, in contrast to MTs in GMW. To compute $\langle z\rangle$ from $\langle x\rangle$, $\langle y\rangle$, and precomputed $[\delta_x\delta_y]$, the

parties first non-interactively sample $[\delta_z]_i$ and locally compute $[\Delta_z]_i = i \cdot \Delta_x\Delta_y + \Delta_x[\delta_y]_i + \Delta_y[\delta_x]_i + [\delta_x\delta_y] + [\delta_z]_i$. Then, $\mathcal{P}_i$ sends $[\Delta_z]_i$ to $\mathcal{P}_{1-i}$, both compute $\Delta_z = [\Delta_z]_0 + [\Delta_z]_1$ and set $([\delta_z]_i, \Delta_z)$ as the output share.

A.3.3 Obtaining Preprocessing Material. The optimizations of the multiplication protocols from [7, 81] use specific forms of correlated randomness to achieve lower communication and round complexity than the original OT-based multiplication protocol of GMW [75]. The idea behind both optimizations is to shift as many computational-heavy parts as possible to an input-independent offline phase. The concrete precomputation protocols are based on either Oblivious Transfer (OT) [5, 56, 91], or Homomorphic Encryption (HE) [29, 41, 87]. We will focus on OT-based protocols because OT has much better communication due to recent breakthroughs in Silent OT [15].

B ADDITIONAL EXPERIMENTS

B.1 Runtime Benchmarks

B.1.1 Runtime of massively parallel circuits (Table 3). As elaborated in §1, massive parallelization is relevant in outsourcing scenarios [51] where the outsourcing servers process the data of many users simultaneously which is heavily used for privacy-preserving machine learning [71, 76, 89, 102].

We present our online runtimes for the parallel AES circuit in the LAN setting, shown in Table 3 which uses a single core and 32 cores in all frameworks. The results reveal a better performance of ABY for circuits with a small SIMD size, but also its unreliability, as we were unable to execute $N_P = 10^6$ parallel circuits due to a segmentation fault. Similarly, MP-SPDZ performed well for small N_P, but likewise failed to execute the largest parallel circuit. Particularly interesting is MOTION, which is 2.77× slower than ABY for small N_P, but handles the execution of the largest circuit without error and is 4.35× faster than SEEC. SEEC's runtime is, depending on the used optimizations, between ABY and MOTION for small N_P. For $N_P = 10^6$, the early deallocation of gates (FG from §4.2) increases the runtime by 1.07× over the baseline (SL).

Our online runtime evaluation shows the drawbacks and benefits of the asynchronous circuit evaluation in MOTION. For scalar circuits or small SIMD sizes, asynchronous evaluation adds significant runtime overhead over evaluation in layers. However, the asynchronous evaluation allows MOTION to evaluate massively parallel circuits faster than SEEC (Table 3). When comparing SEEC with the other frameworks that employ layer-by-layer evaluation, namely ABY and MP-SPDZ, the differences in performance are very minimal. Across all of these three frameworks, multi-core execution does not improve runtime at all and even worsens it a little. This is due to the overhead that comes from multi-core execution, e.g., synchronization and workload distribution across the different cores. Hence, this experiment shows the single benefit of asynchronous evaluation that comes with significant costs (cf. §5.1 and §5.2). When comparing just the single-core performance of MOTION and the other frameworks, the performance gap closes significantly. Only at a SIMD size of 1,000,000, MOTION shows a runtime improvement of up to 26%, whereas for smaller SIMD sizes, SEEC massively improves over MOTION's single-core runtime performance by 8.97× for $N_P = 1$ and still by 1.56× for $N_P = 10^4$.

Table 3: Parallel AES online runtimes in the LAN setting. We report the numbers for single-core performance and multi-core performance with 32 cores. Best results are marked in bold. SEEC optimizations are explained in §4.2.1 (SL), §4.2.4 (FG), and §4.2.2 (IS).

Framework	N_P Cores	1	100	1,000	10,000	100,000	1,000,000
ABY [31]	1	**48.8**	**53.6**	**73.8**	**230.8**	1,386.0	–
	32	60.4	62.4	86.4	314.0	1,537.4	–
MOTION [17]	1	815.4	730.2	969.2	927.6	1,583.6	9,794.8
	32	189.6	186.8	208.2	294.4	**732.0**	**2,594.2**
MP-SPDZ [54]	1	84.4	85.8	124.6	540.4	1,821.2	–
	32	89.4	88.6	139.8	547.0	2,139.8	–
SEEC - SL	1	81.8	90.0	128.0	363.0	1,735.4	13,290.0
	32	76.8	84.8	137.8	607.2	1,916.0	11,295.0
SEEC - DL/FG	1	93.4	111.8	161.8	309.8	1,673.2	14,984.2
	32	103.4	111.4	164.4	655.4	2,197.6	12,060.4

Table 4: Online runtimes in seconds for $N_S = 100$ sequential AES circuits in the LAN setting. Best runtimes per framework are bold.

Framework \ Cores	1	2	8	16	32
ABY [31]	**5.450**	–	–	–	6.684
MOTION [17]	131.352	124.981	20.321	20.966	**18.712**
MP-SPDZ [54]	8.548	–	–	–	**8.527**
SEEC - SL/SC	**5.908**	6.760	7.184	7.159	7.187

Considering applications, MOTION is generally better suited for massive parallelization of the same function inside a big data center. However, when considering virtualization scenarios where MPC parties run on limited compute cores like AWS instances, SEEC is the better choice, as it would improve MOTION's performance in this scenario. SEEC is also the more efficient choice when running MPC on more resource-constrained devices than data centers, like smartphones with limited memory and CPU cores. An interesting future optimization for SEEC would be the addition of an asynchronous execution mode and automatic mode change when beneficial.

B.1.2 Impact of using multiple CPU cores (Table 4). We report the LAN runtimes of sequential AES circuits when utilizing multiple CPU cores in Table 4. For ABY and SEEC, restricting the available number of CPU cores to a single core via `taskset`[10] can result in better performance than when using all available 32 cores. The performance of MOTION highly depends on the available parallelism. Increasing the number of cores decreases runtime substantially. Especially for massively parallel circuits, e.g., $N_P = 10^6$ parallel AES circuits, MOTION is the fastest option.

Notably, the runtimes for ABY and SEEC worsen similarly by around 22% due to multi-core execution. We expect this to be due

[10] https://man7.org/linux/man-pages/man1/taskset.1.html

to ABY's and SEEC's memory layout which uses arena allocation of gates, i.e., gates are stored inside one continuous block of memory to minimize memory consumption. While arena allocation allows for compact memory layout, it does not trivially allow for efficient multi-threading. Especially for scalar circuits, memory accesses incur a high overhead in relation to the actual gate operation and can be negatively affected by multi-threading.

For MP-SPDZ, using multiple cores barely improves performance in this benchmark, however, comparing it with the results in Table 3 (cf. Appendix B.1.1), multi-core execution does not always have a positive impact on the runtimes observed.

Combining our observations with the previous ones from Table 3, it is clear that the asynchronous evaluation method of MOTION is perfectly suited to exploit multi-threading in the best way possible. On the other hand, the layer-by-layer evaluation seems to not profit from multi-threading at all or at least not in the same way. However, even the significant runtime improvements that MOTION enjoys due to asynchronous evaluation do not come for free. As discussed in §5.1, MOTION has a very high memory overhead due to the `boost::fibers` and a significant single-core runtime overhead.

B.2 Communication

We give a comparison of the communication, as reported by the frameworks, in Table 5. Although the number of AND gates is the same for a sequential or parallel circuit of the same size, the actual communication can differ. This effect is largest for MOTION, where the communication for 100 sequential AES circuits is 76.48× higher than the parallel version. This overhead is a factor 1.88× for SEEC, and 1.37× for ABY. The self-reported communication of MP-SPDZ is nearly identical between both versions of the circuit. However, this is in part due to a difference in the way the communication metrics are collected. ABY, MOTION, and SEEC track the number of bytes written to a socket, which includes any overhead introduced by serialization schemes but omits TCP/IP headers. In contrast, MP-SPDZ tracks only the payload as communication, i.e., the shares $[e]_i$, $[d]_i$ for GMW (Appendix A.3.1), hence ignoring overheads on the application layer and of TCP/IP. When applicable, the use of SIMD operations cannot only reduce memory consumption and runtime but also the communication due to a more efficient encoding.

Table 5: Online communication in MB for evaluating N_P parallel or N_S sequential AES circuits. Best results are marked in bold.

Framework	N_P or N_S Circuit	1	100	10,000
ABY [31]	Par.	0.002	**0.153**	15.259
	Seq.	0.002	0.209	20.943
MOTION [17]	Par.	0.635	0.830	15.870
	Seq.	0.635	63.477	–
MP-SPDZ [54]	Par.	**0.002**	0.159	**15.259**
	Seq.	0.002	0.154	15.411
SEEC	Par.	0.010	0.180	17.186
	Seq.	0.004	0.339	33.853

Table 6: Online runtime in ms and communication of SEEC with static layer iteration (SL) for ABY2.0 [81] compared to GMW [39] for different Bristol circuits.

Circuit	#ANDs	#Rounds	Protocol	Online Communication (B)	Online Runtime (ms)	Peak Memory
mul8	70	6	GMW	184	2	953.31 kB
			ABY2.0	119	3	953.31 kB
mul16	286	10	GMW	354	4	953.31K
			ABY2.0	221	4	953.57K
mul32	1118	13	GMW	669	6	953.31K
			ABY2.0	468	7	1.47M
mul64	4350	16	GMW	1678	11	1.10M
			ABY2.0	930	10	3.64M
FP-eq	315	9	GMW	333	3	953.57K
			ABY2.0	215	3	963.95K
FP-lt	381	67	GMW	1903	32	953.57K
			ABY2.0	1308	29	1.01M
FP-ceil	650	71	GMW	2003	10	953.31K
			ABY2.0	1470	28	1.17M
FP-f2i	1467	94	GMW	2933	41	953.57K
			ABY2.0	1926	46	1.67M
FP-i2f	2416	206	GMW	6090	94	1.01M
			ABY2.0	4129	92	2.29M
FP-add	5385	235	GMW	7667	106	1.26M
			ABY2.0	4975	101	4.12M
FP-mul	19626	129	GMW	9089	63	2.07M
			ABY2.0	4842	59	12.31M
FP-div	82269	3619	GMW	120615	1687	6.82M
			ABY2.0	78224	1752	48.40M
FP-sqrt	91504	6507	GMW	200487	3174	8.99M
			ABY2.0	133653	3106	59.63M
AES-128	6400	60	GMW	3646	36	1.82M
			ABY2.0	1990	32	5.02M
AES-192	7168	72	GMW	4058	45	1.96M
			ABY2.0	2226	41	5.22M
AES-256	8832	84	GMW	5012	52	2.22M
			ABY2.0	2691	47	7.30M
Keccak-f	38400	24	GMW	11448	70	6.13M
			ABY2.0	5232	59	29.10M
SHA-256	57947	1607	GMW	50393	800	5.01M
			ABY2.0	32851	802	17.77M

B.3 Additional Circuits

We benchmark SEEC using both GMW and ABY2.0 with additional Bristol circuits [2]. We report the online communication, runtimes and peak memory consumptions for both protocols in Table 6.

As expected, the online communication almost halves when evaluating the circuits using ABY2.0 compared to GMW while having similar runtimes. However, we also observe a higher peak memory consumption for ABY2.0 which is due to the setup phase. The setup phase for ABY2.0 computes all necessary multiplication masks (cf. Appendix A.3.2) by running GMW multiplication on the input masks, which, in turn, currently does not make use of sub-circuit iteration and, unfortunately, currently resorts to inlining sub-circuits. We leave incorporating a similar sub-circuit iteration algorithm (cf. §4.1.1) for the preprocessing phase as future work.